



《模式识别》

第二章 特征提取与表示

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.



课程目录（暂定）

❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	3学时
❑	第十一章	循环神经网络	3学时
❑	第十二章	Transformer	3学时
❑	第十三章	自监督与半监督学习	3学时
❑	第十四章	开放世界模式识别	6学时



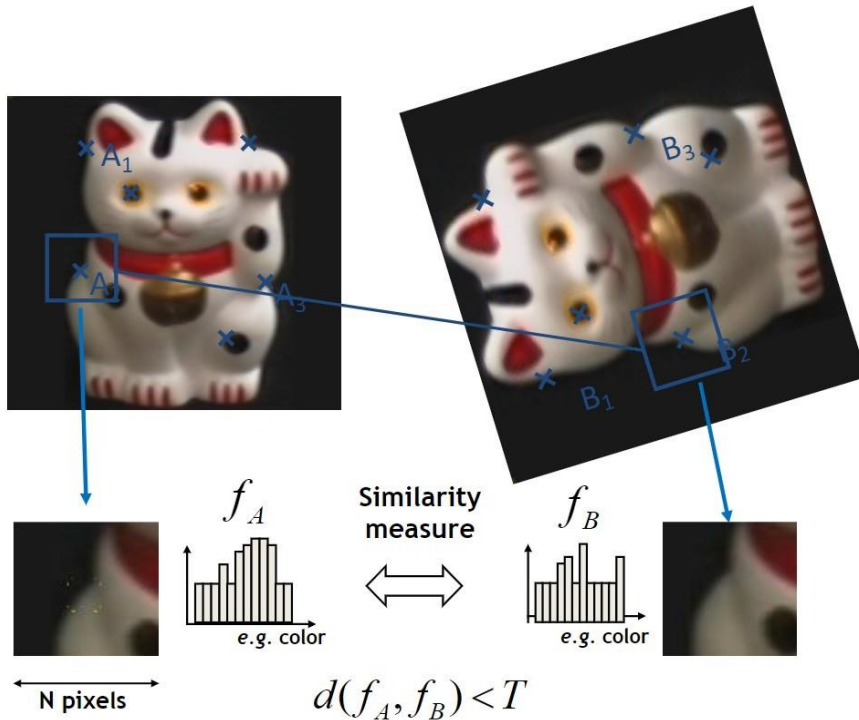
第二部分：特征表示



What will we learn today?

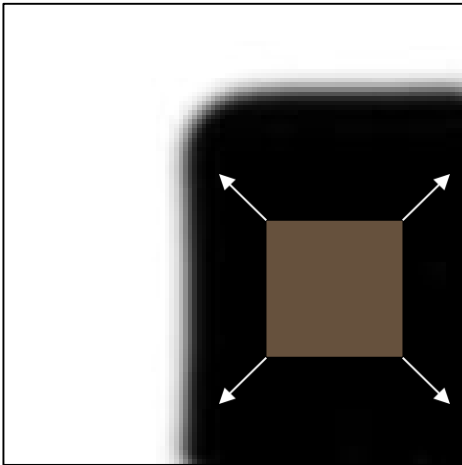
- A quick review
- Scale invariant region detection
 - Automatic scale selection
 - Difference-of-Gaussian (DoG) detector
- SIFT: an image region descriptor
- HoG: another image region descriptor
- Application: Panorama

Quick review : General Approach

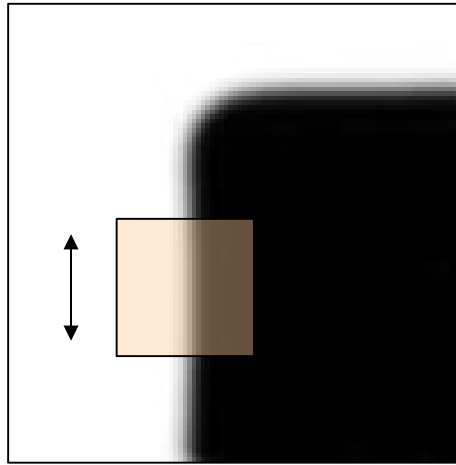


1. Find a set of distinctive key-points
2. Define a region around each keypoint
3. Extract and normalize the region content
4. Compute a local descriptor from the normalized region
5. Match local descriptors

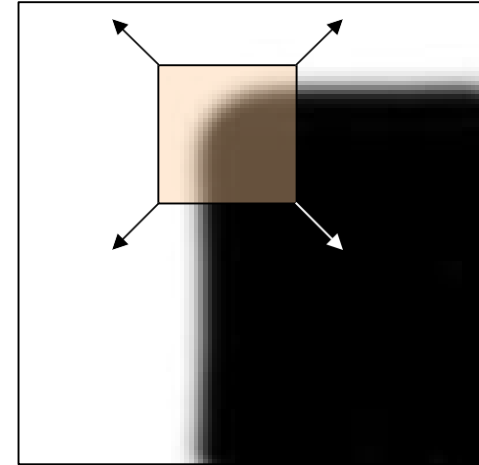
Quick review: Harris Corner Detector



“flat” region:
no change in
all directions



“edge”:
no change along
the edge direction



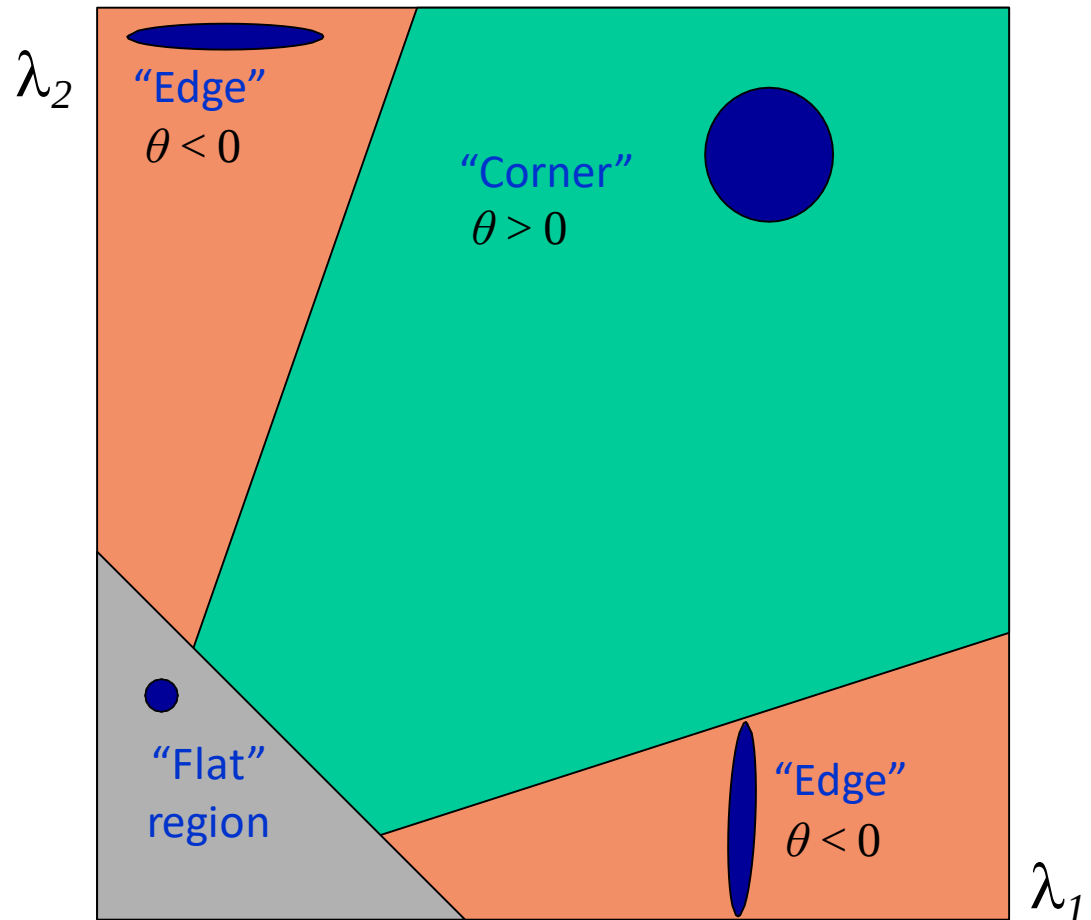
“corner”:
significant change
in all directions

Quick review: Corner Response Function

$$\theta = \det(M) - \alpha \text{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

Fast approximation
[Harris and Stephens,
1988]

- Avoid computing the eigenvalues
- α : constant
(0.04 to 0.06)

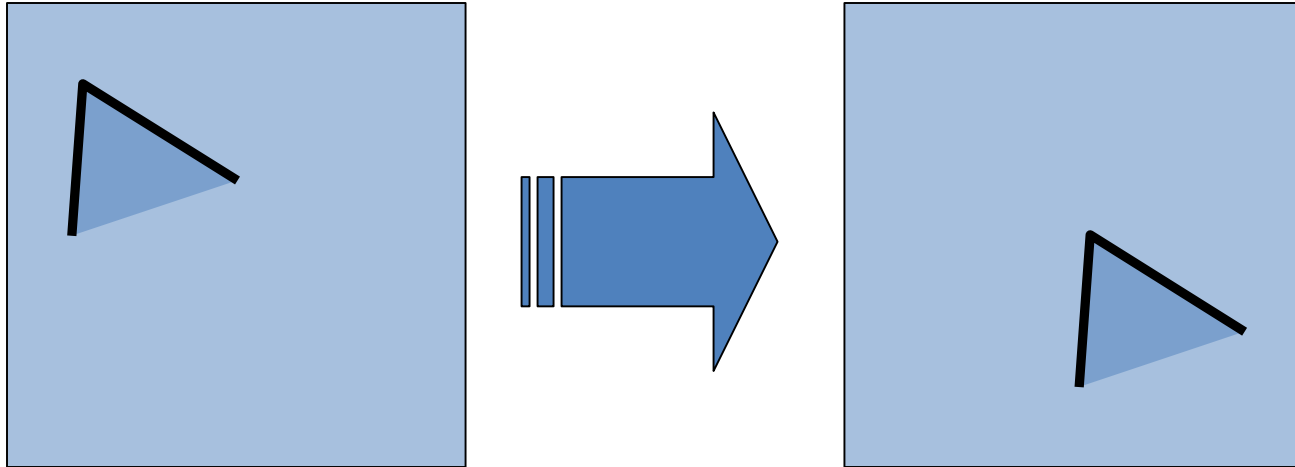


Quick review: Workflow

- Input Image
- Compute corner response function θ
- Take only the local maxima of θ , where $\theta > \text{threshold}$



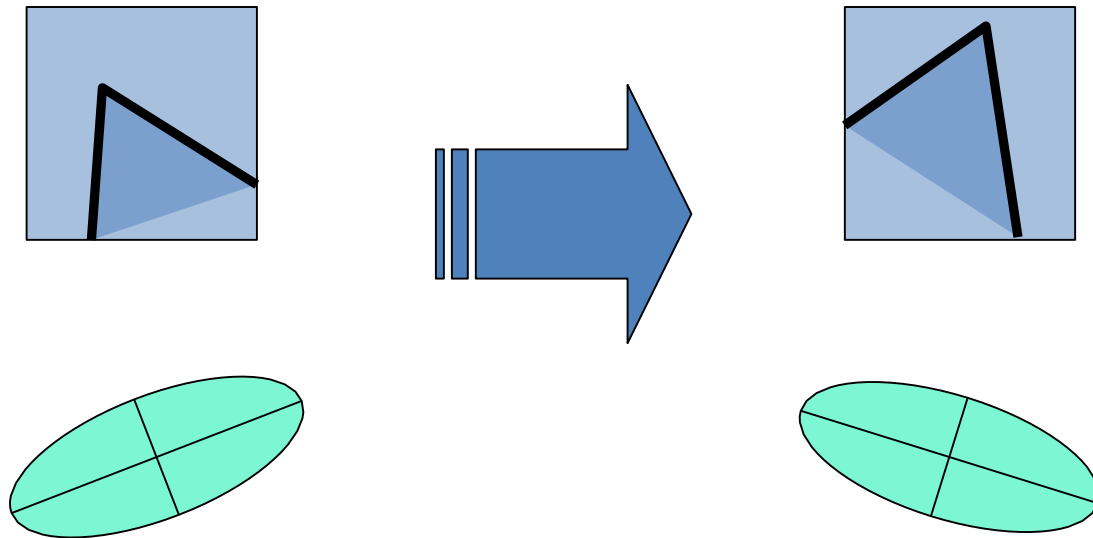
Quick review: Harris Corner Detector



Derivatives and window function are equivariant
w.r.t. translation

Corner location is equivariant w.r.t. translation

Quick review: Harris Corner Detector



Second moment (autocorrelation) ellipse rotates but its shape (i.e. eigenvalues) remains the same

Corner location is equivariant w.r.t. image rotation

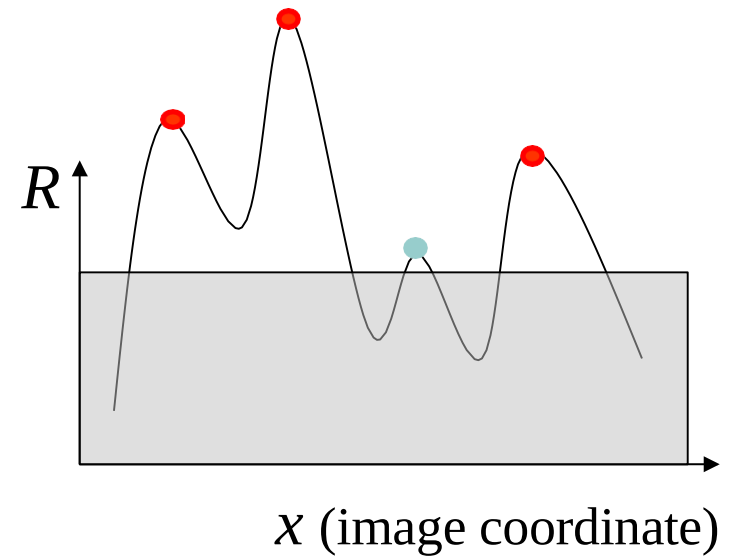
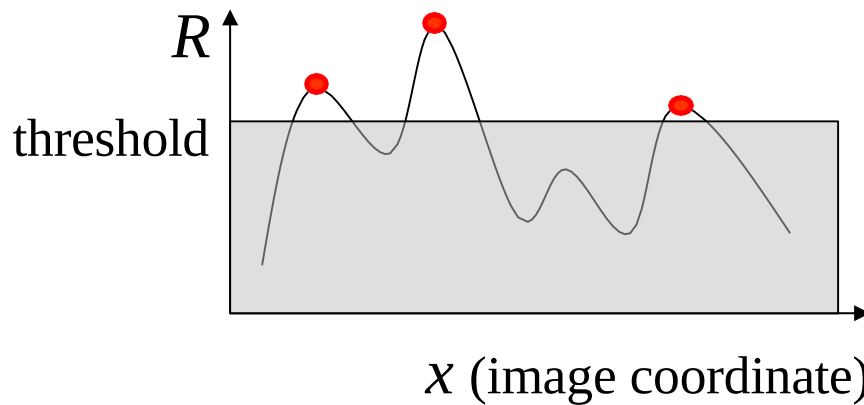
Quick review: Harris Corner Detector



$$I \rightarrow a I + b$$

Only derivatives are used $\Rightarrow$ invariance to intensity shift $I \rightarrow I + b$

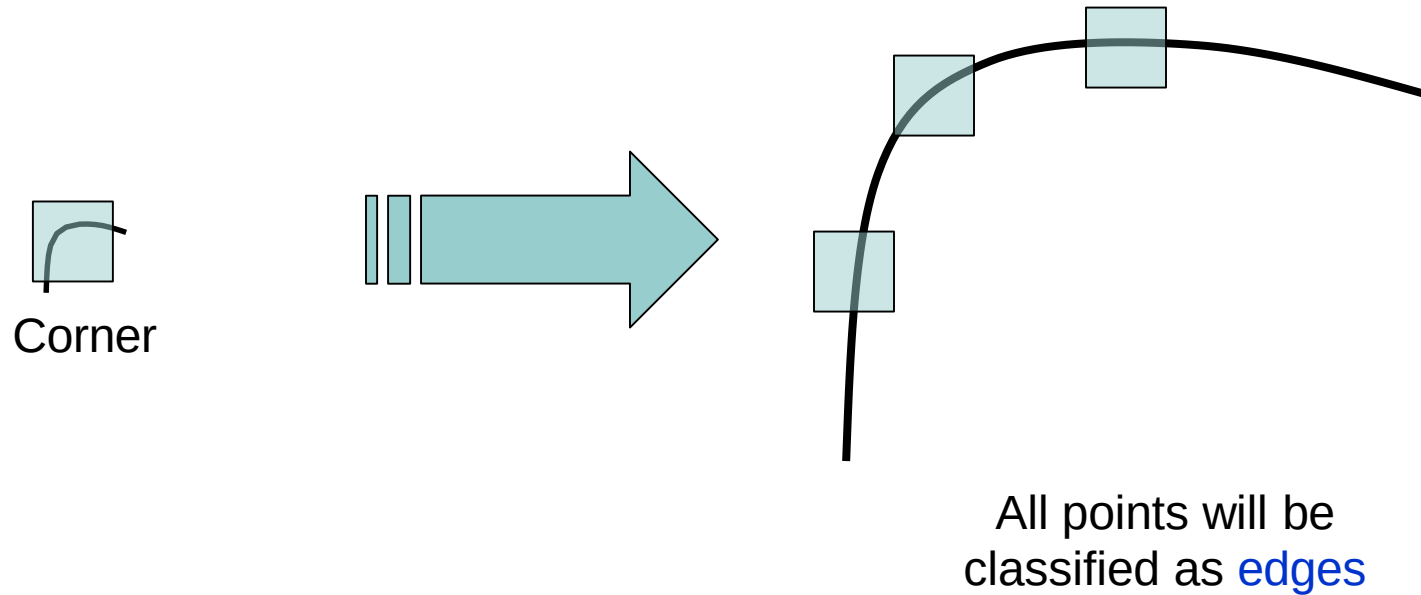
- Intensity scaling: $I \rightarrow a I$



Partially invariant to affine intensity change

Quick review: Harris Corner Detector

- Scaling



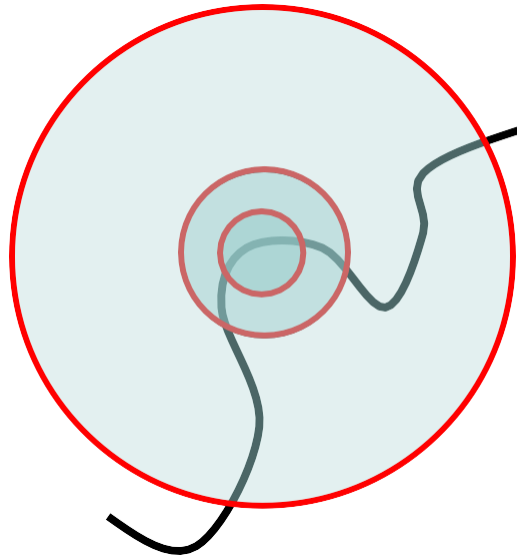
Neither invariant nor equivariant to scaling

What we will learn today?

- A quick review
- Scale invariant region detection
 - Automatic scale selection
 - Difference-of-Gaussian (DoG) detector
- SIFT: an image region descriptor
- HoG: another image region descriptor
- Application: Panorama

Scale invariant detection

Suppose you're looking for corners



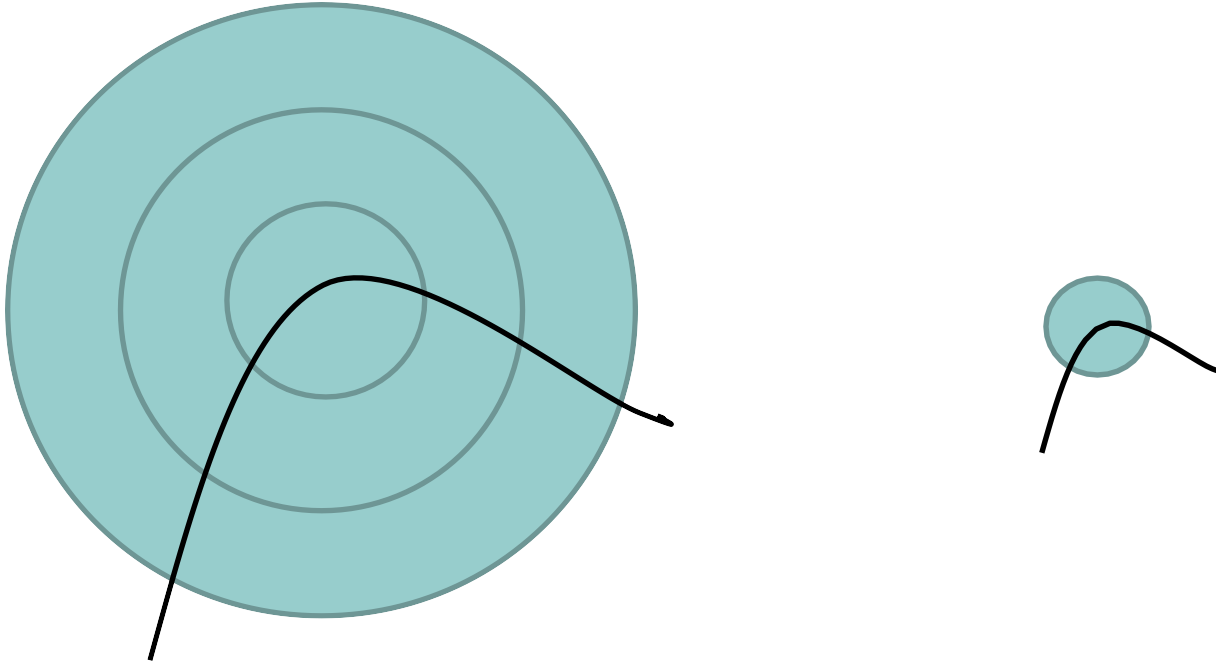
Key idea: find scale that gives local maximum of f

- in both position and scale
- One definition of f : the Harris operator

Scale invariant detection

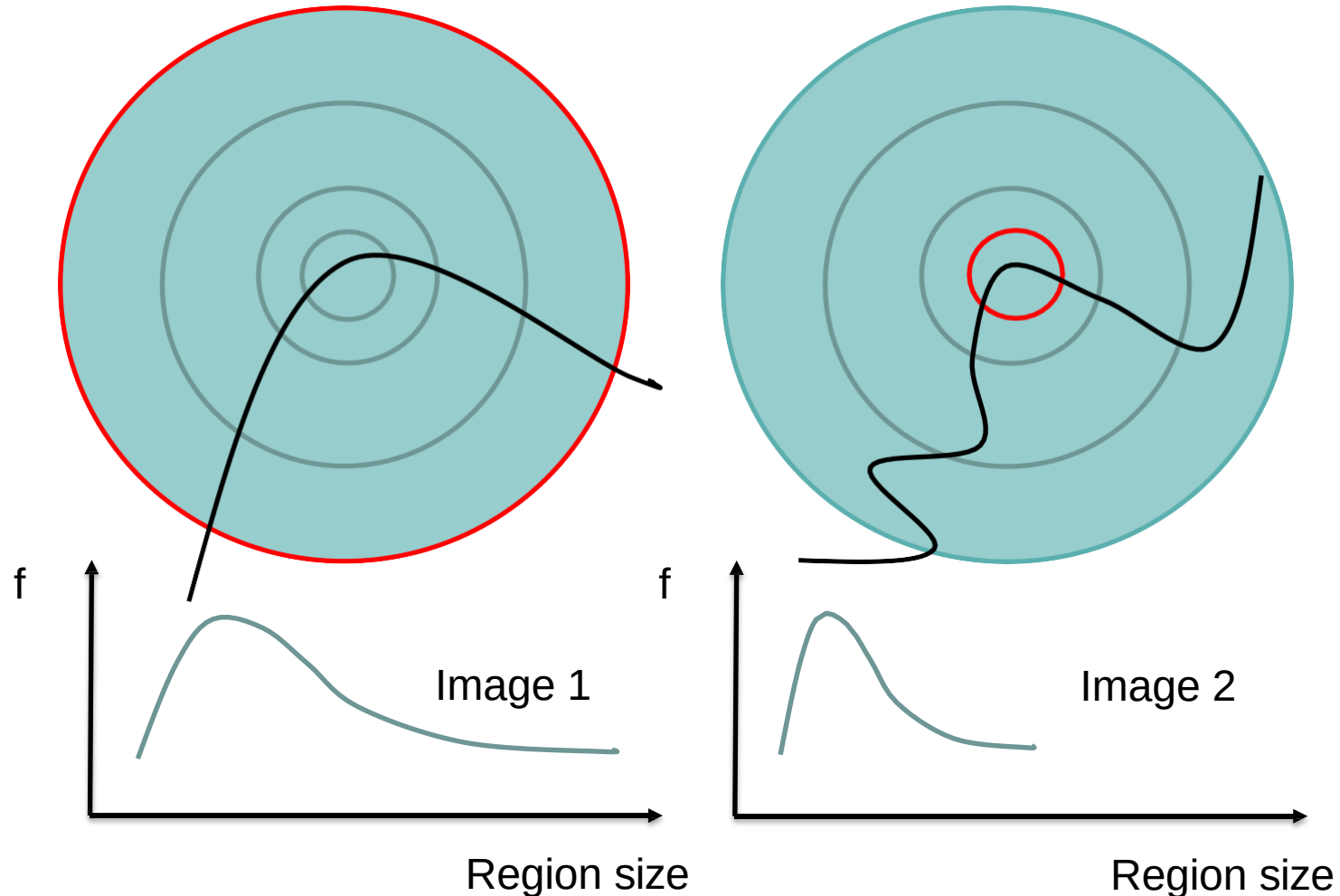
- Scale Invariant Detection

- Consider regions (e.g. circles) of different sizes around a point
- What region size do we choose, so that the regions look the same in both images?



Scale invariant detection

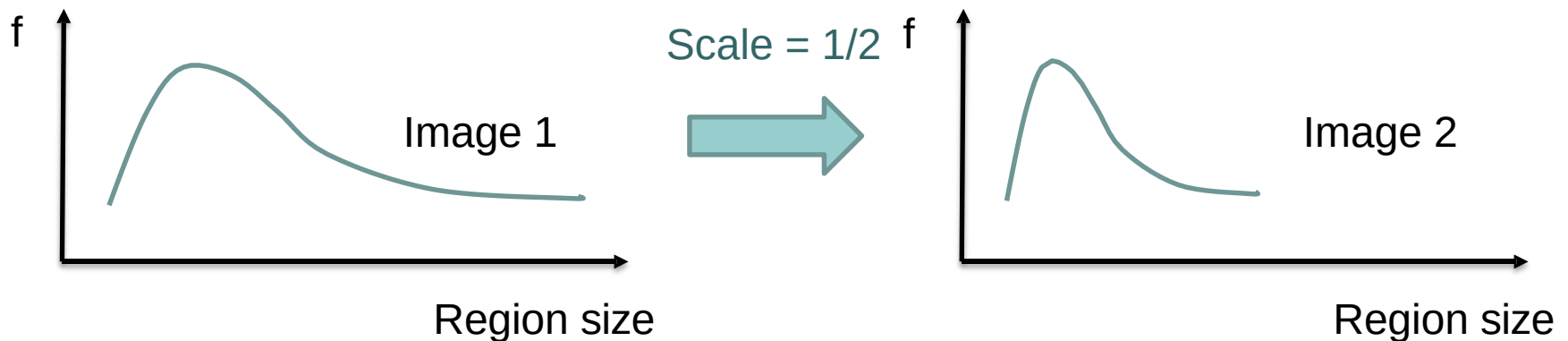
- The problem: how do we choose corresponding circles **independently** in each image?



Scale invariant detection

- Scale Invariant Detection Solution:
 - Design a function on the region (circle), which is “scale invariant” (equivariant w.r.t. scale, i.e., the same for corresponding regions, even if they are at different scales)

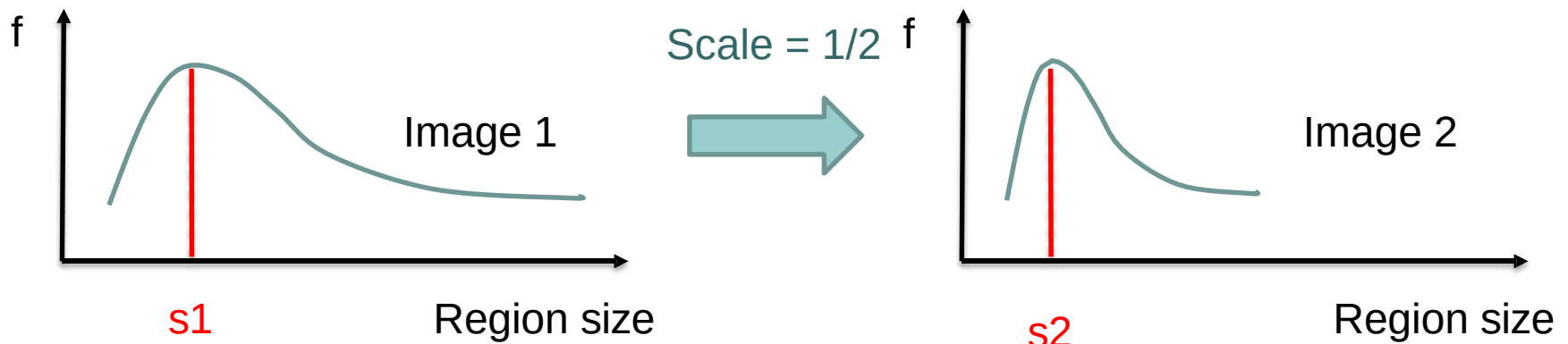
Example: average intensity. For corresponding regions (even of different sizes) it will be the same.
 - For a point in one image, we can consider it as a function of region size (circle radius)



Scale invariant detection

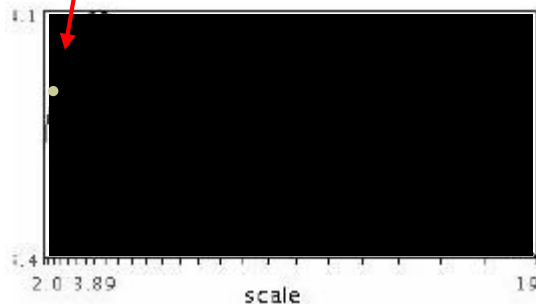
- Common approach:
 - Take a local maximum of this function
- Observation: region size, for which the maximum is achieved, should be equivariant with image scale.

Important: this scale invariant region size is found in each image independently!

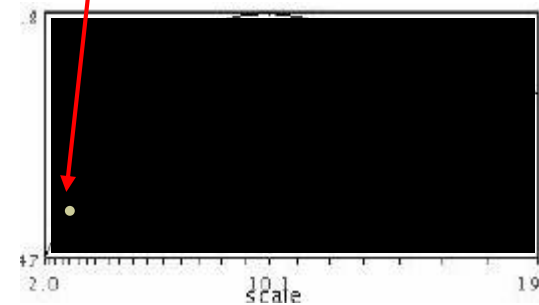


Automatic Scale Selection (K. Grauman, B. Leibe)

- Function responses for increasing scale (scale signature)



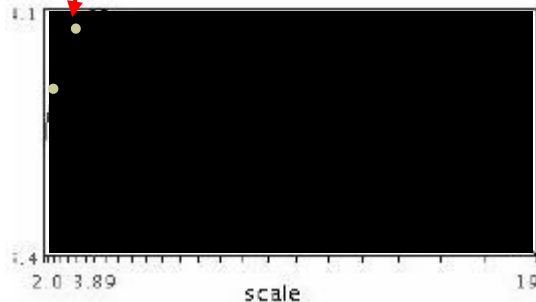
$$f(I_{i_1...i_m}(x, \sigma))$$



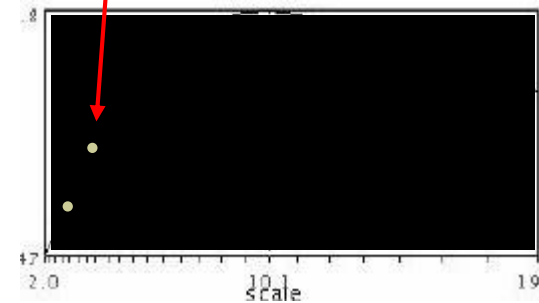
$$f(I_{i_1...i_m}(x', \sigma))$$

Automatic Scale Selection

- Function responses for increasing scale (scale signature)



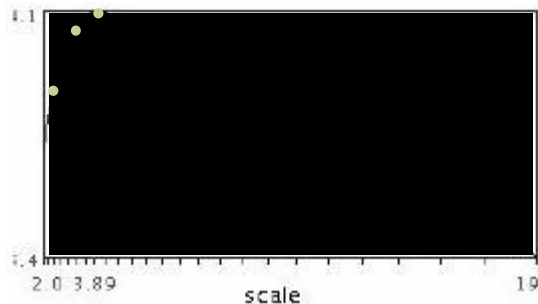
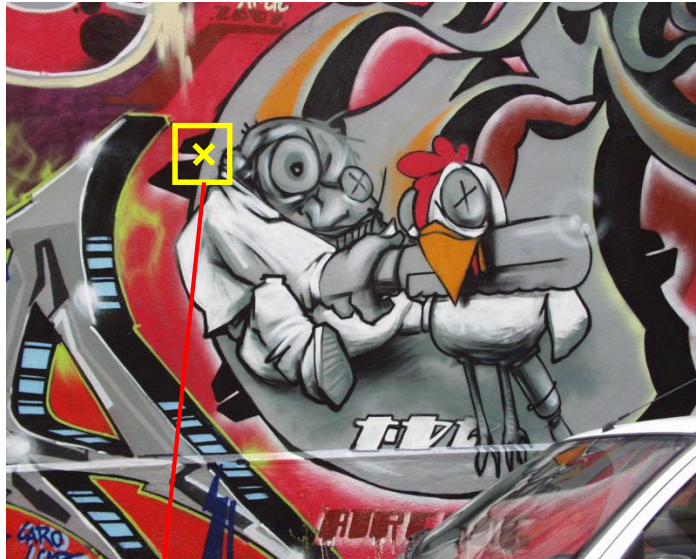
$$f(I_{i_1...i_m}(x, \sigma))$$



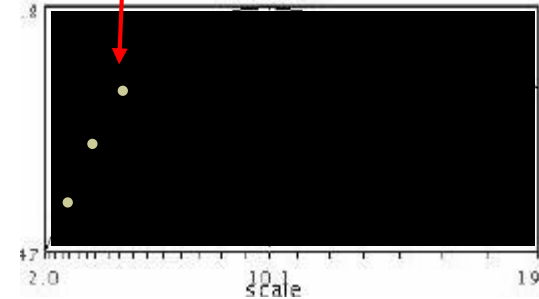
$$f(I_{i_1...i_m}(x', \sigma))$$

Automatic Scale Selection

- Function responses for increasing scale (scale signature)



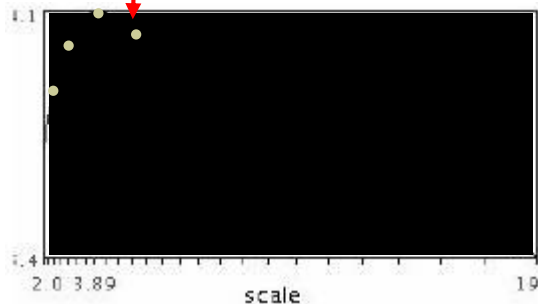
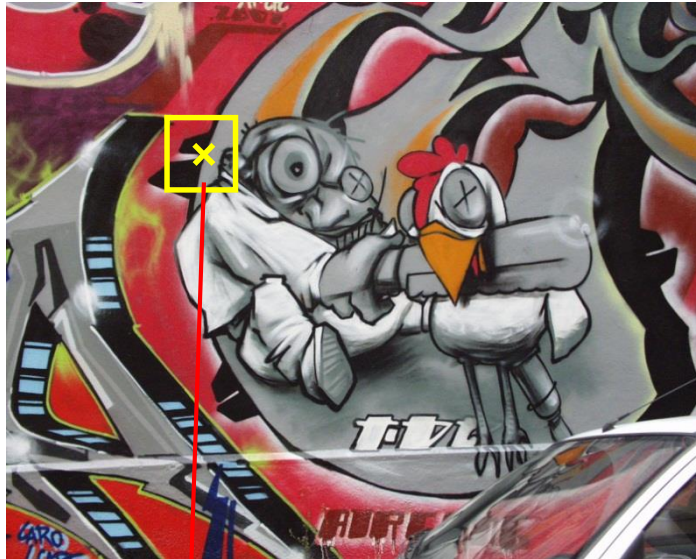
$$f(I_{i_1...i_m}(x, \sigma))$$



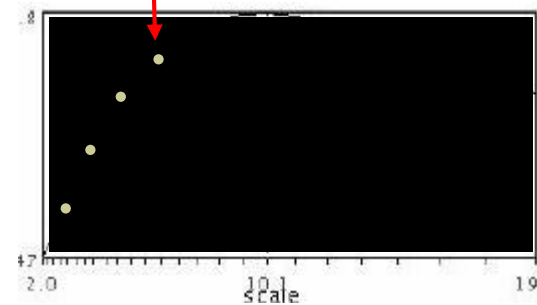
$$f(I_{i_1...i_m}(x', \sigma))$$

Automatic Scale Selection

- Function responses for increasing scale (scale signature)



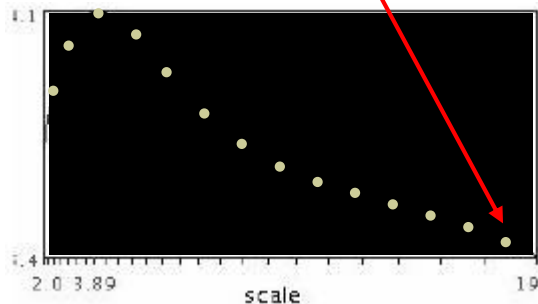
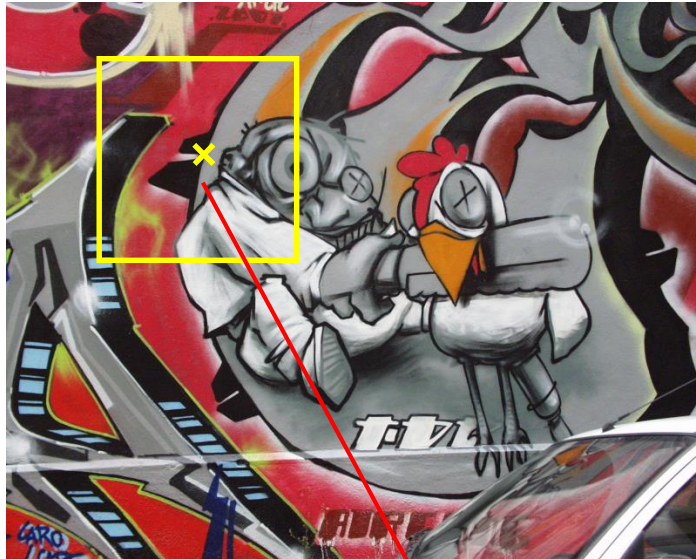
$$f(I_{i_1 \dots i_m}(x, \sigma))$$



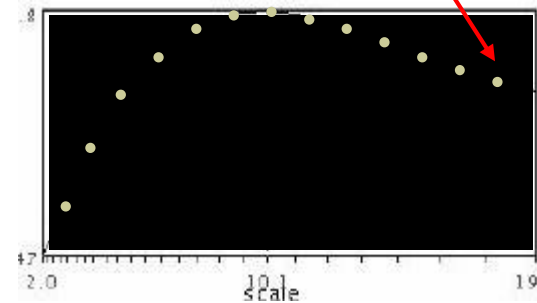
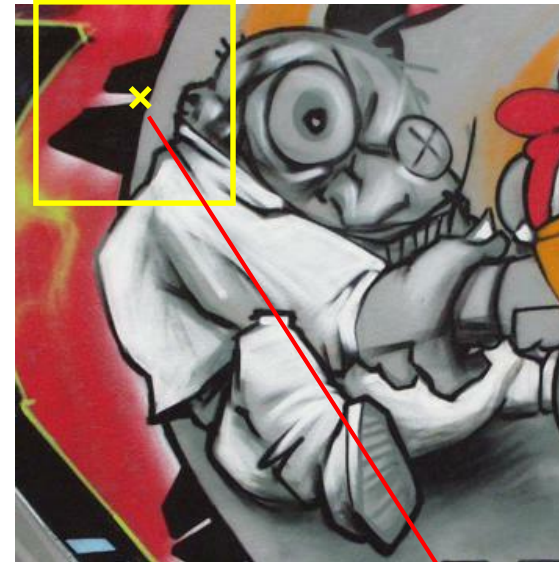
$$f(I_{i_1 \dots i_m}(x', \sigma))$$

Automatic Scale Selection

- Function responses for increasing scale (scale signature)



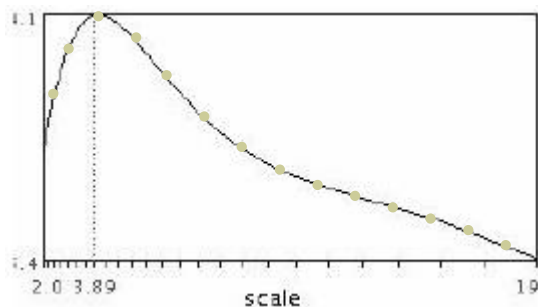
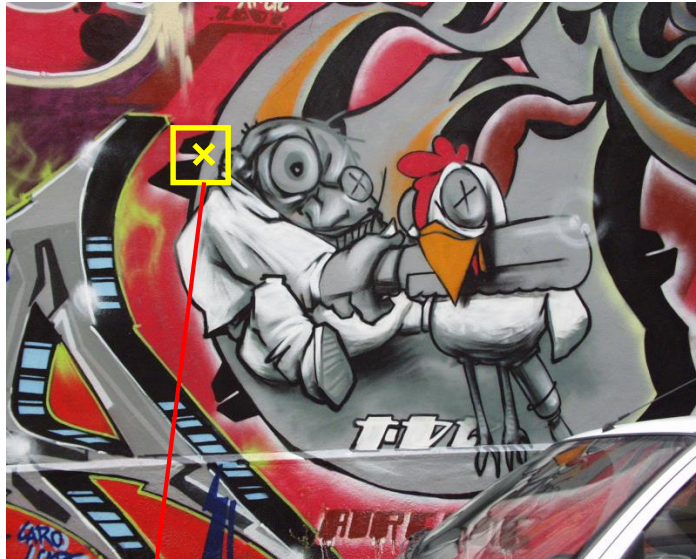
$$f(I_{i_1...i_m}(x, \sigma))$$



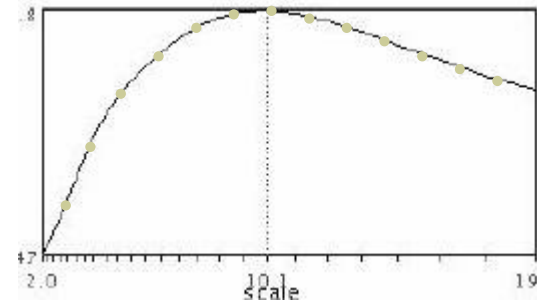
$$f(I_{i_1...i_m}(x', \sigma))$$

Automatic Scale Selection

- Function responses for increasing scale (scale signature)



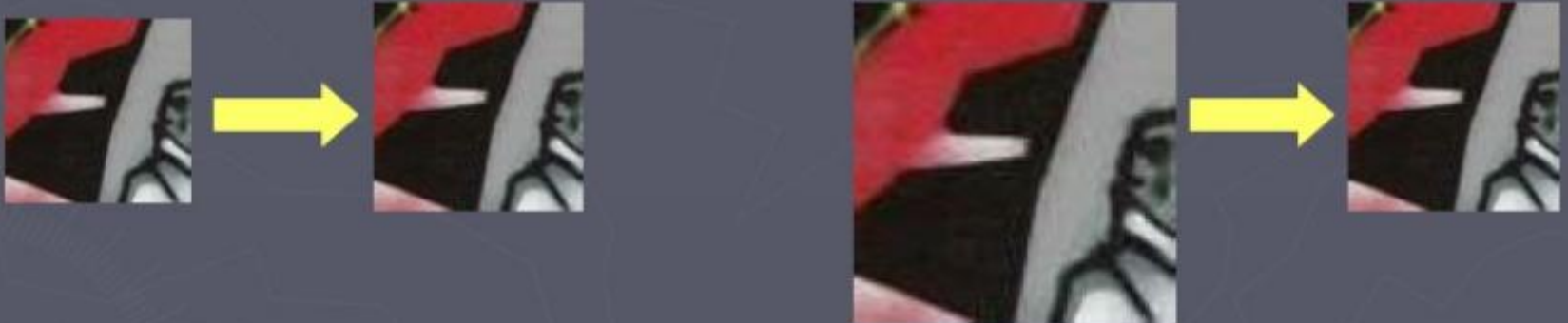
$$f(I_{i_1...i_m}(x, \sigma))$$



$$f(I_{i_1...i_m}(x', \sigma'))$$

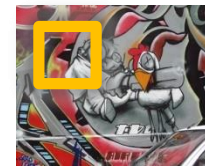
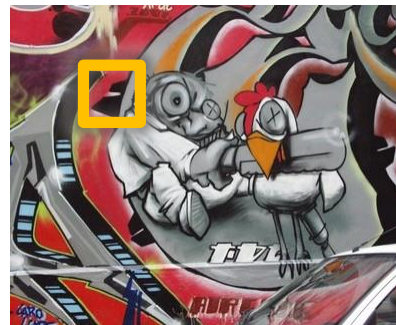
Automatic Scale Selection

Normalize: rescale to fixed size



Implementation

- Instead of computing f for larger and larger windows, we can implement using a fixed window size with a Gaussian pyramid

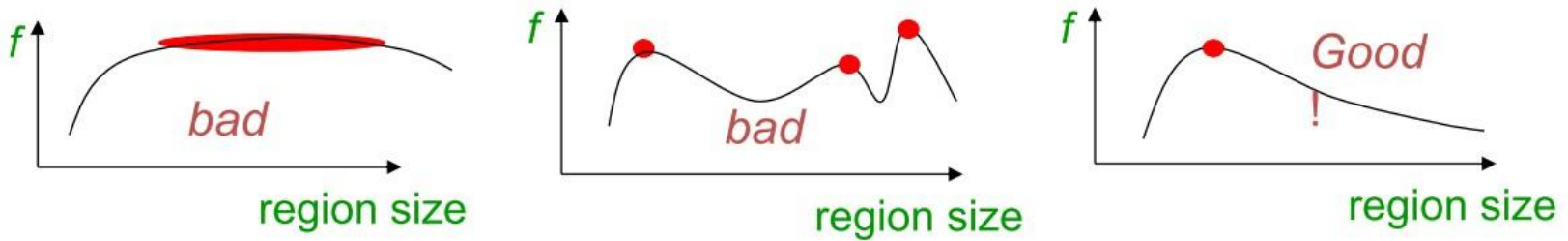


(sometimes need to create in-between levels, e.g. a $\frac{3}{4}$ -size image)

Scale invariant detection

- Scale Invariant Detection

- A “good” function for scale detection: has one stable sharp peak



- For usual images: a good function would be one which responds to contrast (sharp local intensity change)

Scale invariant detection

- We will use functions for determining scale that have the form of

$$f = \text{Kernel} * \text{Image}$$

Kernel 1: Normalized Laplacian of Gaussians (LoG)

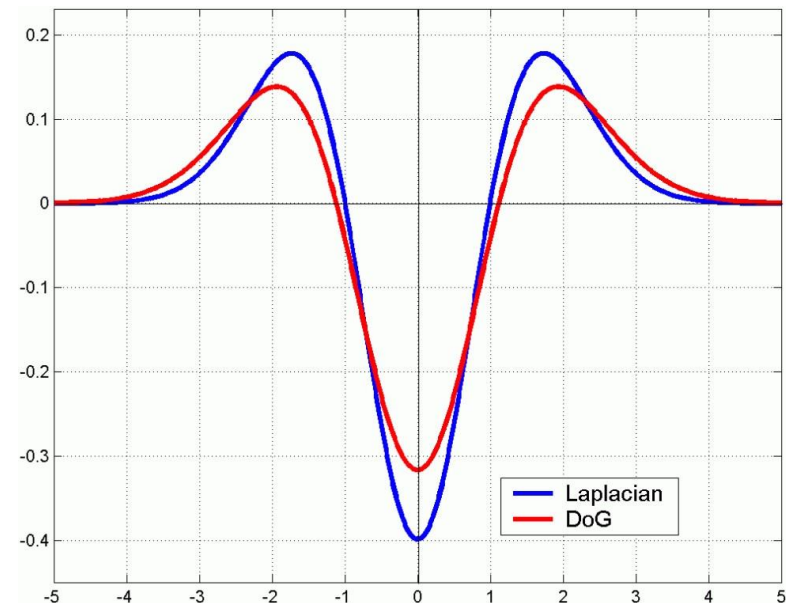
$$L = \sigma^2 (G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma)) = \sigma \frac{\partial G}{\partial \sigma} \approx (k - 1) \text{DoG}$$

Kernel 2: Difference of Gaussians (DoG)

$$\text{DoG} = G(x, y, k\sigma) - G(x, y, \sigma)$$

where G is Gaussian

$$G(x, y, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2+y^2}{2\sigma^2}}$$



Note: both kernels are invariant (equivariant) to *scale* and *rotation*

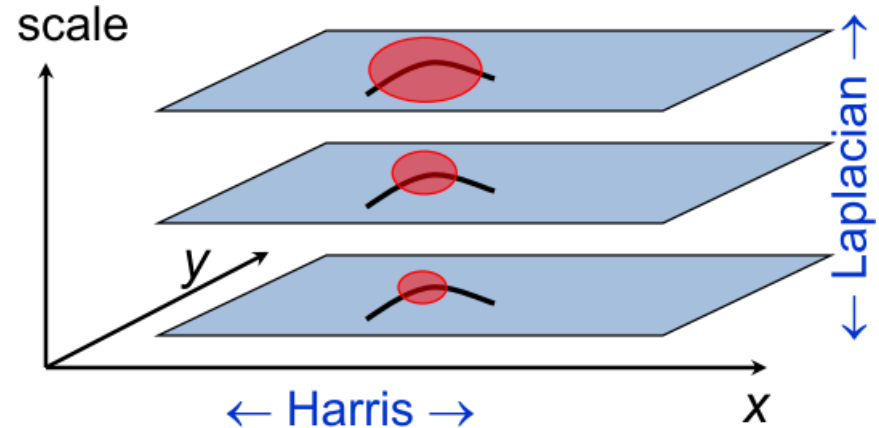
Scale invariant detectors

- Scale Invariant Detectors

- Harris-Laplacian*

Find local extrema of:

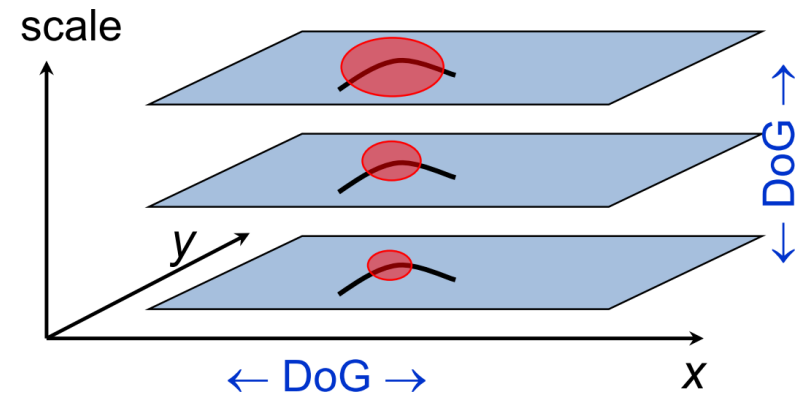
- Harris corner detector (maxima) in space (image coordinates)
- Laplacian in scale



- DoG (from SIFT by Lowe)*

Find local extrema of:

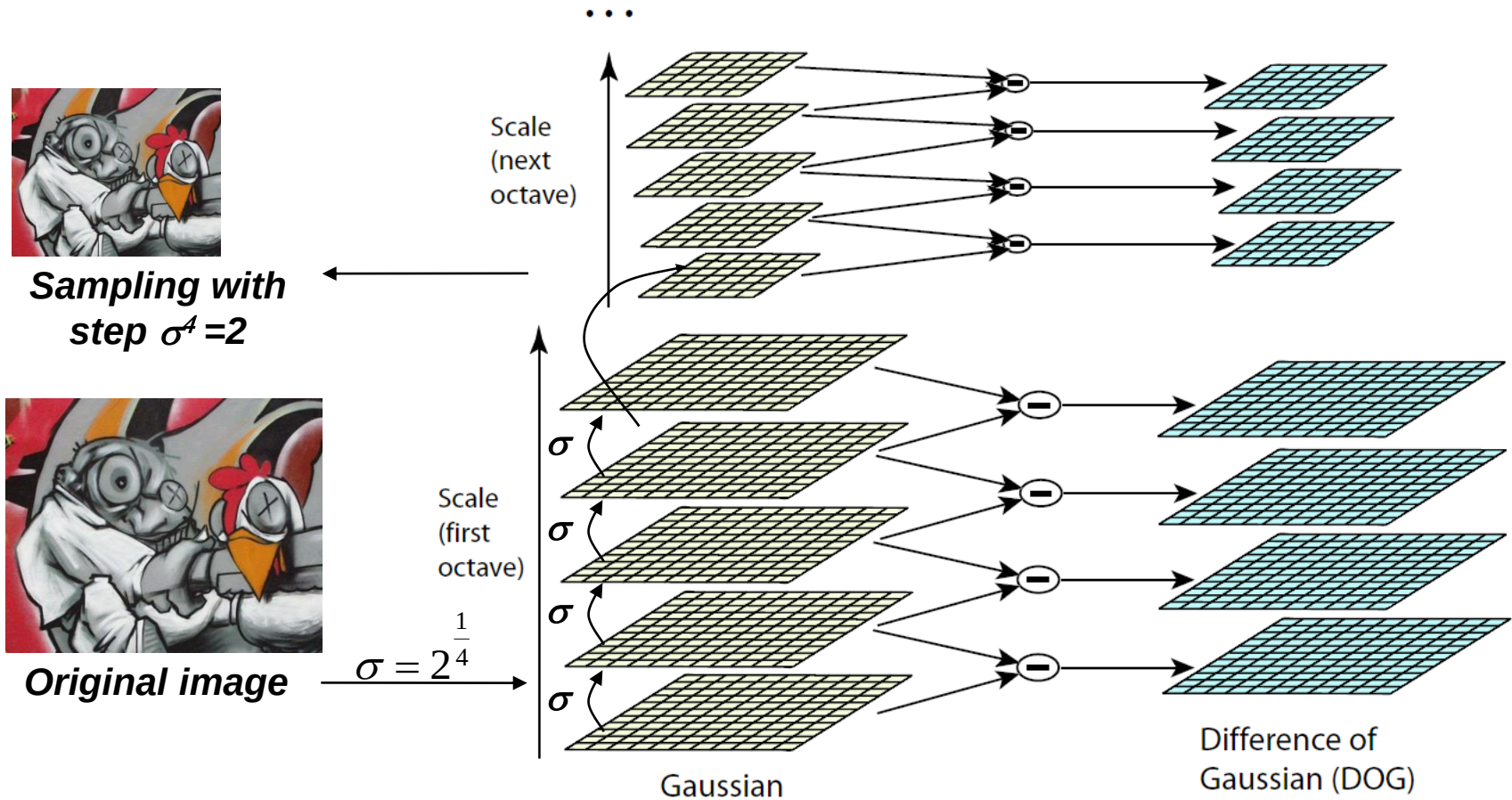
- Difference of Gaussians in space and scale



1 K.Mikolajczyk, C.Schmid. "Indexing Based on Scale Invariant Interest Points". ICCV 2001

2 D.Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". IJCV 2004

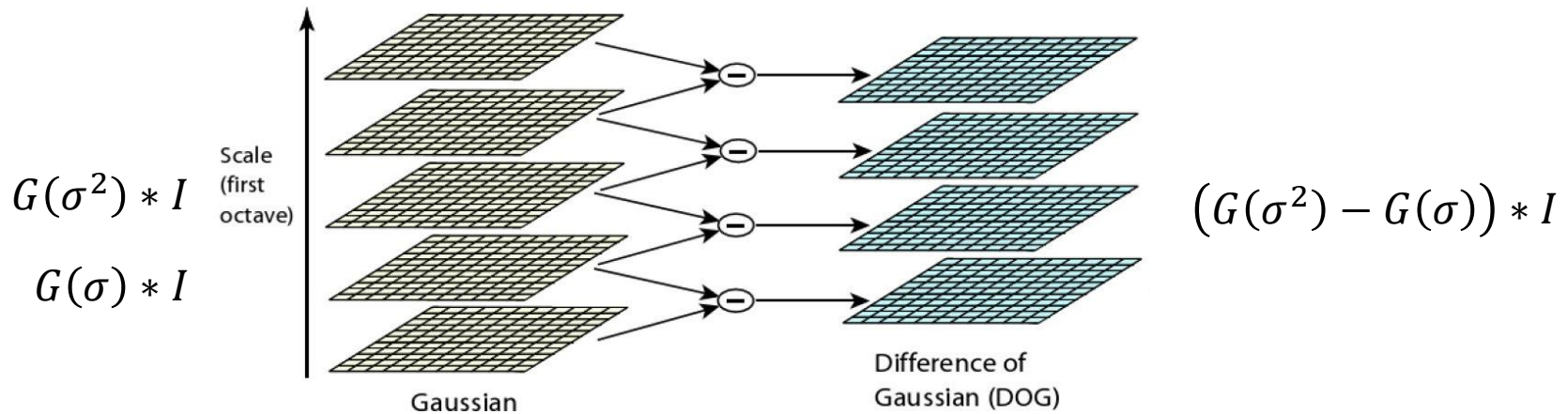
Scale-Space Construction



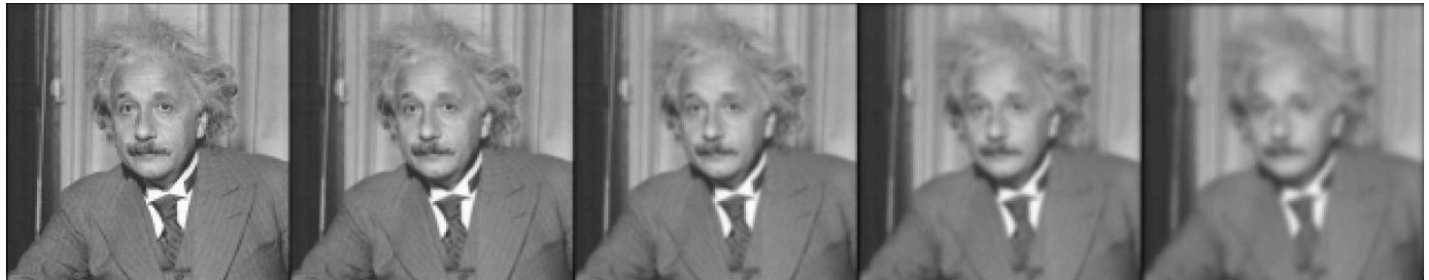
K. Grauman, B. Leibe

Scale invariant detectors

- Difference-of-Gaussians



Gaussian:

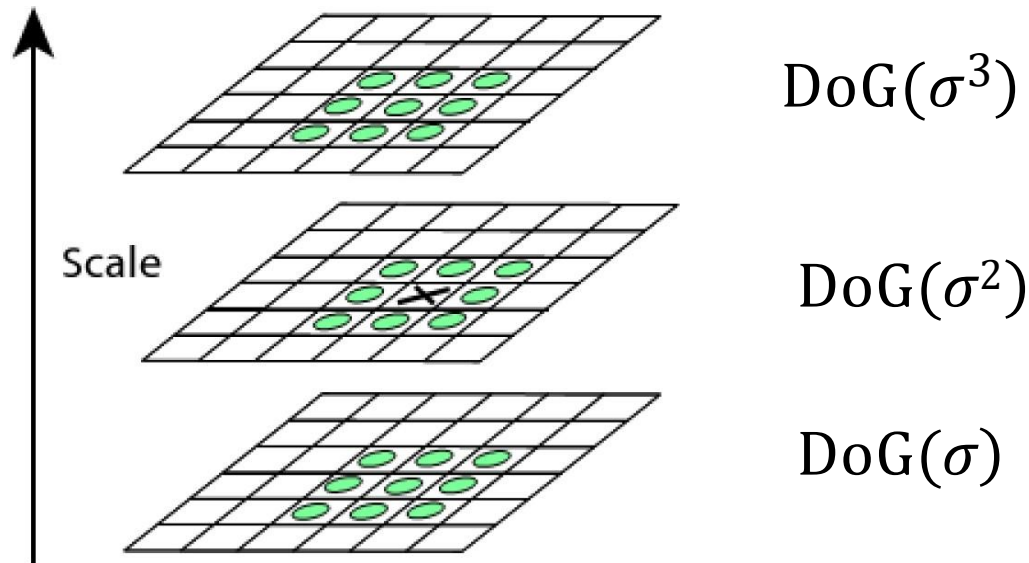


DoG:



Scale invariant detectors

- Scale-Space Extrema
 - Choose all extrema within 3x3x3 neighborhood.



X is selected if it is larger or smaller than all 26 neighbors

Scale invariant detectors

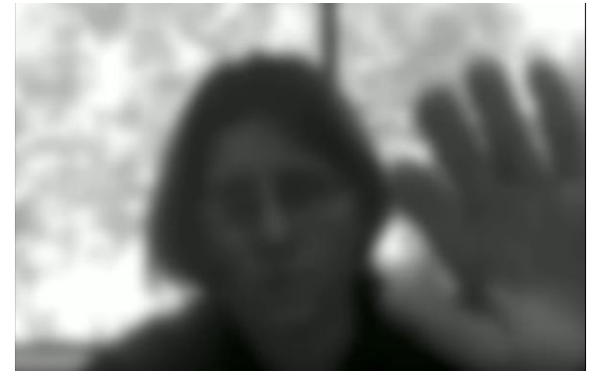
- Example of Gaussian kernel



Original video



Blurred with a gaussian kernel



Blurred with a different gaussian kernel

What happens if you subtract one blurred image from another?

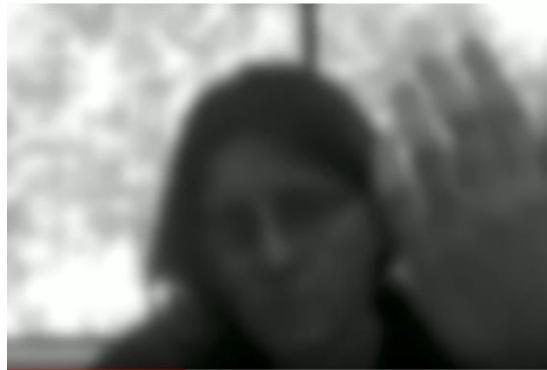
Source: https://www.youtube.com/watch?v=oTud1De_W4s

Scale invariant detectors

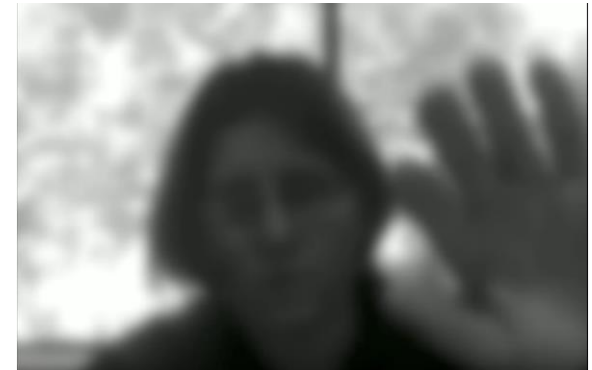
- Difference of Gaussians (DoG)



Original video



Blurred with a gaussian kernel k_1



Blurred with a different gaussian kernel k_2



DoG: $k_2 - k_1$



DoG: $k_3 - k_2$



DoG: $k_4 - k_3$

Scale invariant detectors

- Difference of Gaussians (DoG)

At different resolutions of kernel size, we see different fine details of the image. In other words, we can capture keypoints at varying scales.



DoG: $k_2 - k_1$



DoG: $k_3 - k_2$



DoG: $k_4 - k_3$

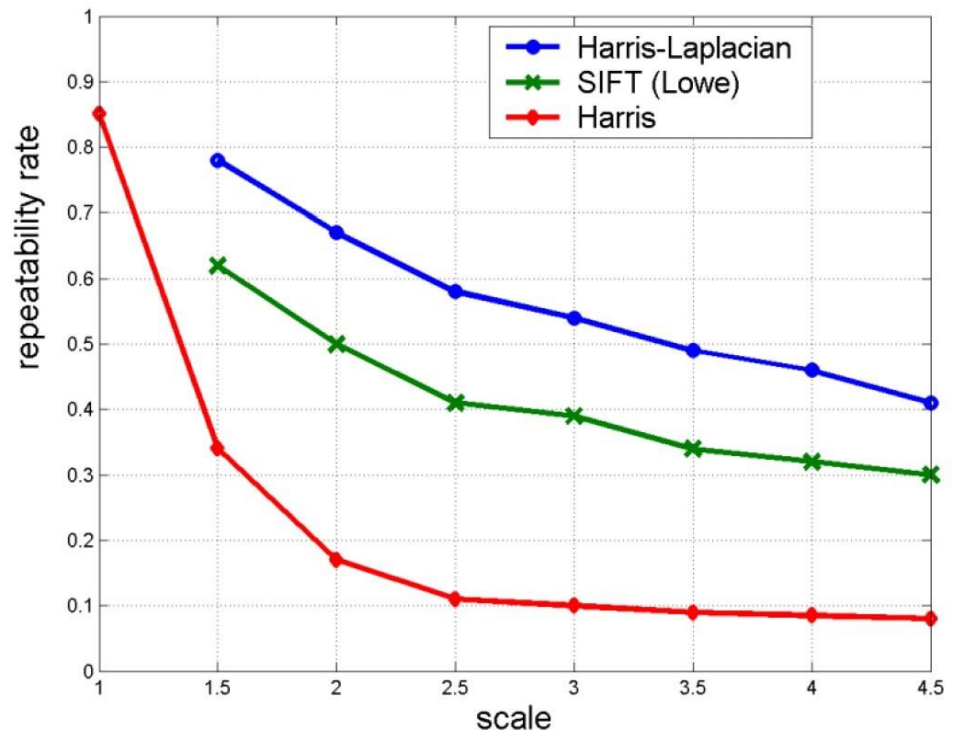
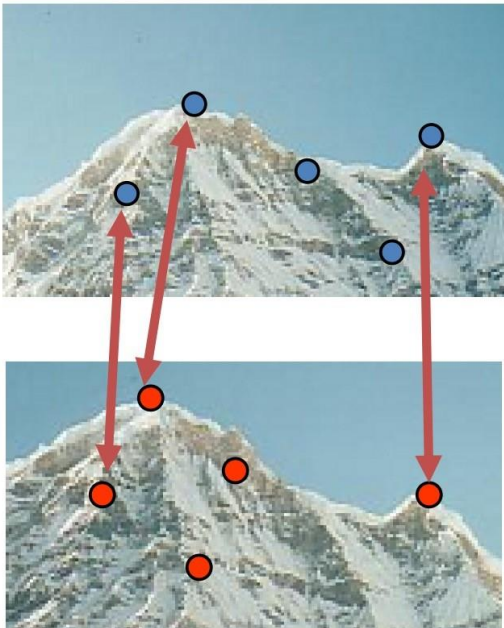
Scale invariant detection

- Scale Invariant Detectors

- Experimental evaluation of detectors w.r.t. scale change

Repeatability rate:

$$\frac{\text{correspondences}}{\text{possible correspondences}}$$



- Harris (corner): accurate center but poor scale
- LoG, DoG (blob): less accurate center but better scale
- Complementary locations used in conjunction

Scale invariant detection

- Scale Invariant Detection: Summary

- Given: two images of the same scene with a large scale difference between them
- Goal: find the same interest points independently in each image
- Solution: search for extrema of suitable functions in scale and in space (over the image)

Methods:

1. Harris-Laplacian [Mikolajczyk, Schmid]: extrema of Laplacian over scale, maximize Harris' measure of corner response over the image
2. SIFT [Lowe]: extrema of Difference of Gaussians over scale and space

Summary

- Scale invariant keypoint detection
 - Automatic scale selection
 - Harris-Laplace detector
 - Difference-of-Gaussian (DoG) detector

What's next?

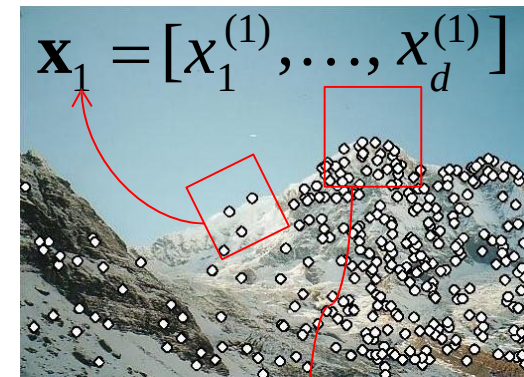
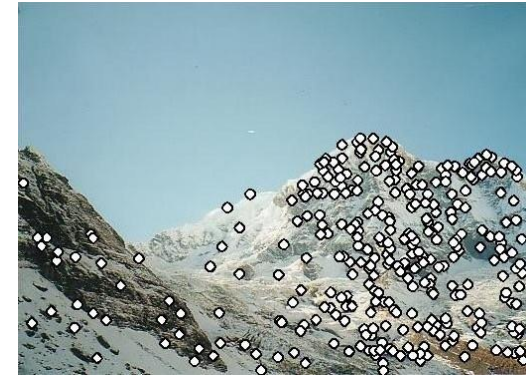
- So we can detect keypoints at varying scales. But what can we do with those keypoints?
- Things we would like to do:
 - Search:
 - We would need to find similar key points in other images
 - Panorama
 - Match keypoints from one image to another.
 - Etc...
- For all such applications, we need a way of `describing` the keypoints.

What we will learn today?

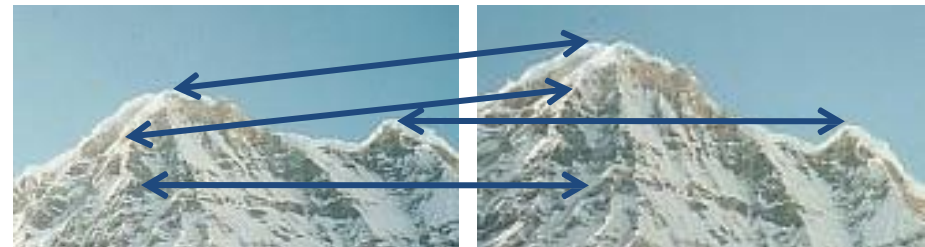
- Scale invariant keypoint detection
 - Automatic scale selection
 - Difference-of-Gaussian (DoG) detector
- **SIFT: an image region descriptor**
- HoG: another image region descriptor
- Application: Panorama

Local features: main components

- 1) Detection: Identify the interest points
- 2) **Description: Extract vector feature descriptor surrounding each interest point.**
- 3) Matching: Determine correspondence between descriptors in two views

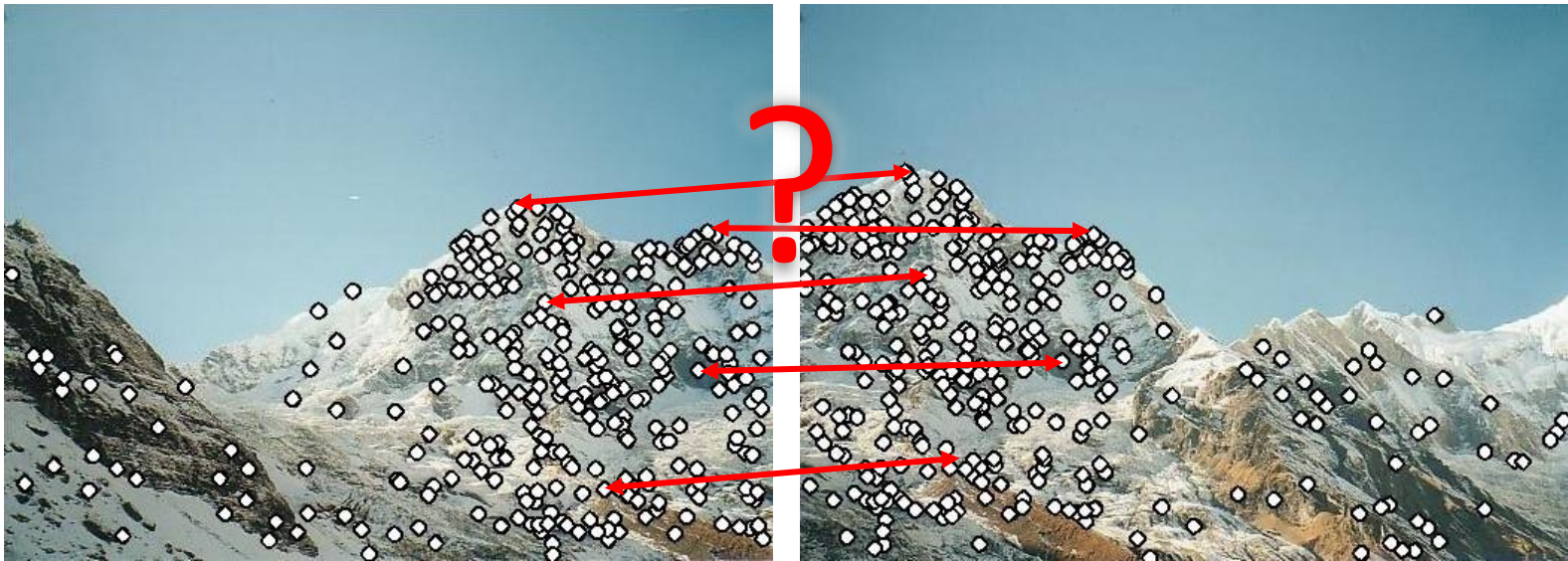


$$\mathbf{x}_2 = [x_1^{(2)}, \dots, x_d^{(2)}]$$



Feature descriptors

- ❑ We know how to detect good points
- ❑ Next question: **How to describe them for matching?**
- ❑ Descriptor: Vector that summarizes the content of the keypoint neighborhood
 - ✓ Point descriptor should be: 1. Invariant, 2. Distinctive



- Simple option: match square windows around the point?
- Better option: SIFT
 - David Lowe, UBC <http://www.cs.ubc.ca/~lowe/keypoints/>

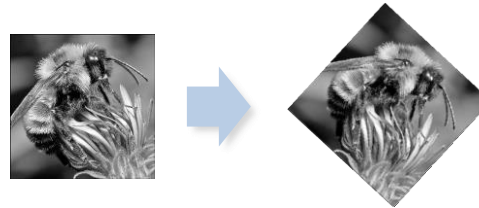
Invariant *and* Distinctive

- Invariant:
 - Descriptor shouldn't change even if image is transformed
- Distinctive (Discriminative):
 - Descriptor should be highly unique for each point

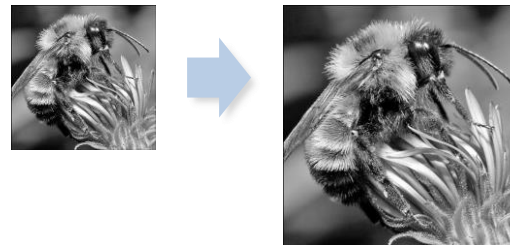
Image transformations revisited

- Geometric

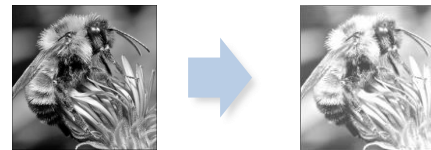
Rotation



Scale



- Photometric
Intensity change



How to achieve invariance

Need both of the following:

1. Make sure your detector is invariant
2. Design an invariant feature descriptor
 - Simplest descriptor: a single 0
 - What's this invariant to?
 - Next simplest descriptor: a square, axis-aligned 5x5 window of pixels
 - What's this invariant to?
 - Let's look at some better approaches...

Rotation invariance for feature descriptors

- Find dominant orientation of the image patch
 - E.g., given by $\mathbf{x}_{\max}$, the eigenvector of the auto-correlation matrix $\mathbf{M}$ corresponding to $\lambda_{\max}$ (the *larger* eigenvalue)
 - Or simply the orientation of the (smoothed) gradient
 - Rotate the patch according to this angle



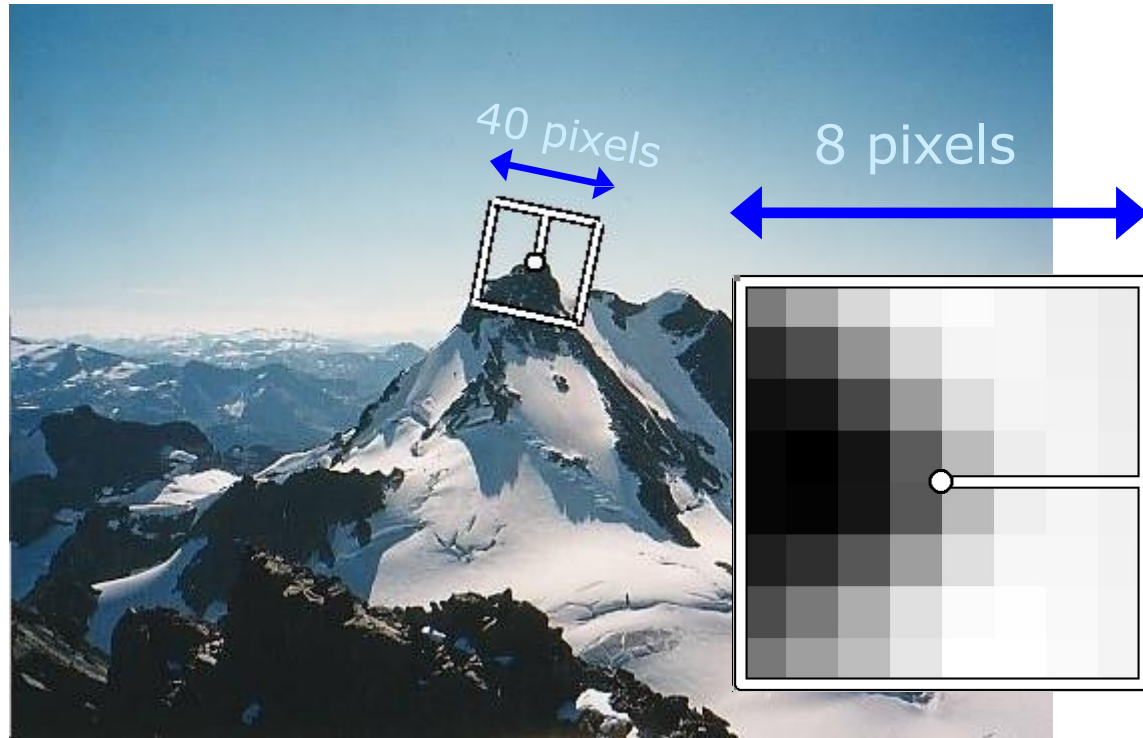
Figure by Matthew Brown

Multiscale Oriented PatcheS descriptor

Approach 1: Normalize patches by rotating them

Take 40x40 square window around detected feature

- Scale to 1/5 size (using prefiltering)
- Rotate to horizontal
- Sample 8x8 square window centered at feature
- Intensity normalize the window by subtracting the mean, dividing by the standard deviation in the window (why?)



Adapted from slide by Matthew Brown

Detections at multiple scales

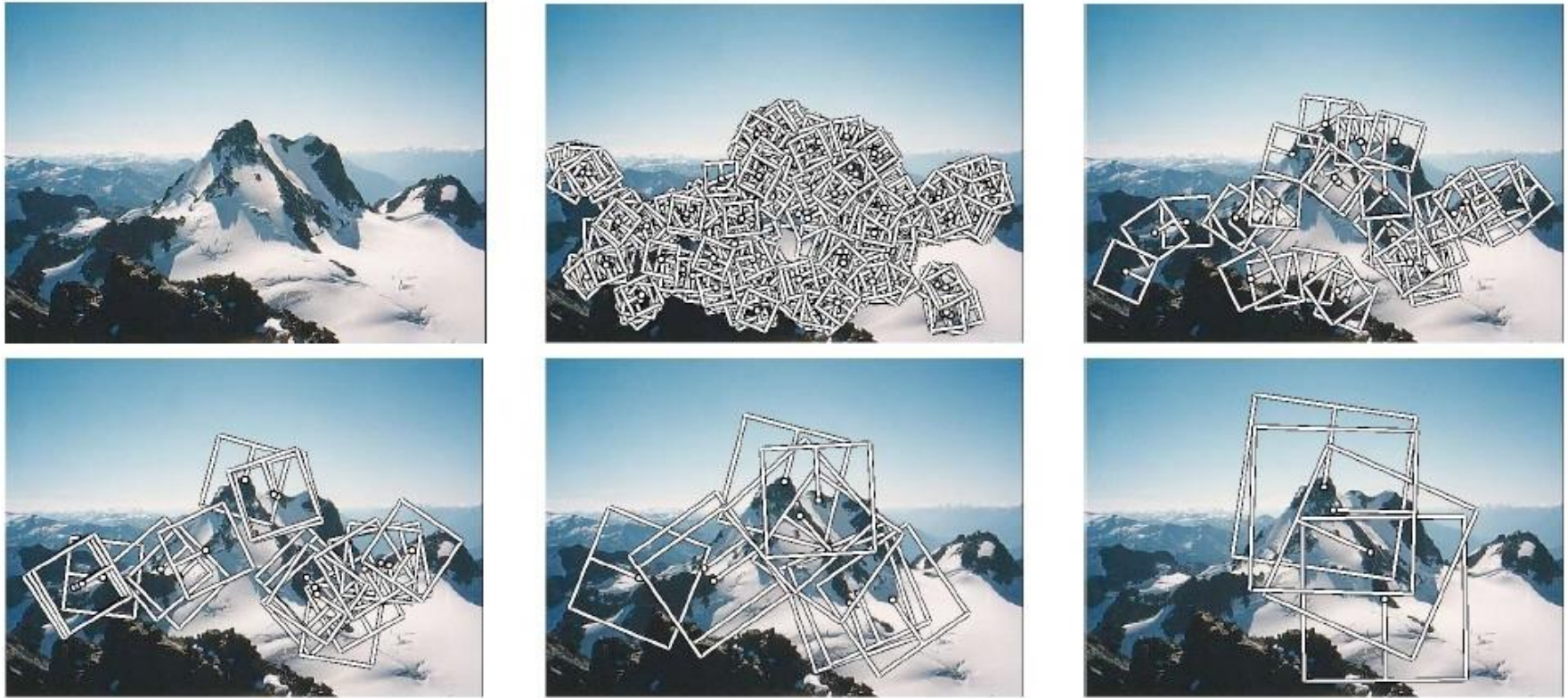
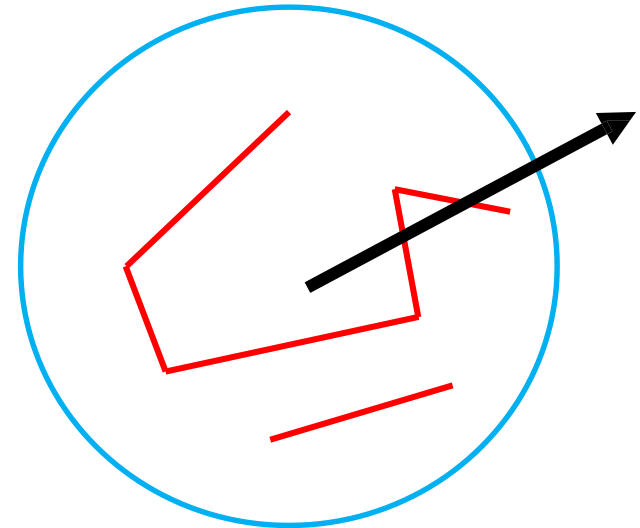


Figure 1. Multi-scale Oriented Patches (MOPS) extracted at five pyramid levels from one of the Matier images. The boxes show the feature orientation and the region from which the descriptor vector is sampled.

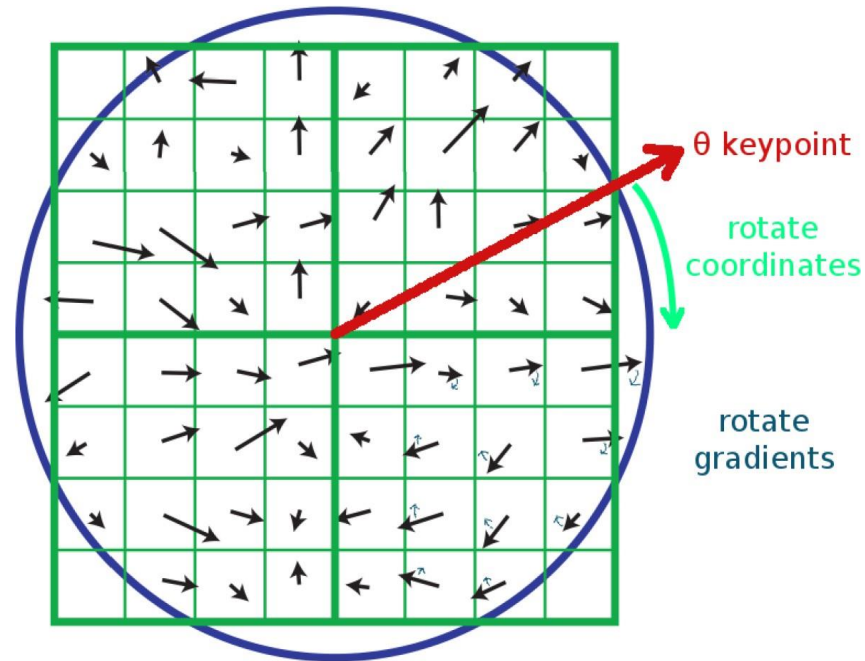
Rotation invariant descriptors

- Approach 2: Construct a rotation invariant descriptor
 - We are given a keypoint and its scale from DoG
 - We will select a characteristic orientation for the keypoint (based on the most prominent gradient there; discussed next slide)
 - We will describe all features relative to this orientation
 - Causes features to be rotation invariant!
 - If the keypoint appears rotated in another image, the features will be the same, because they're relative to the characteristic orientation



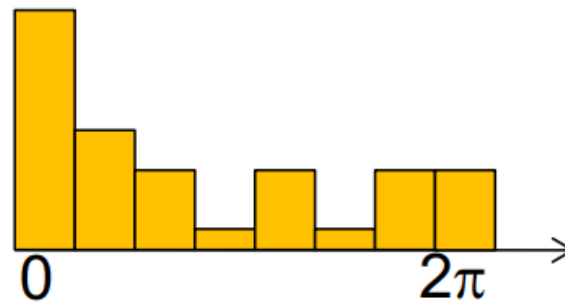
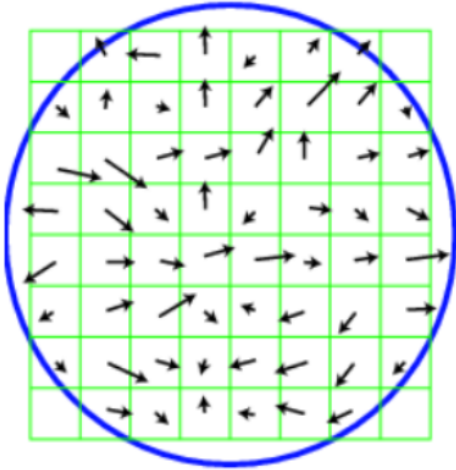
SIFT (Scale-Invariant Feature Transform)

- Gradient-based descriptor to capture texture in the keypoint neighborhood
- Use the blurred image associated with the keypoint's scale (not window size)
- Take image gradients over the keypoint neighborhood (e.g. 16×16)
- To become rotation invariant, rotate the gradient directions AND locations by (keypoint orientation)
 - Now we've cancelled out rotation and have gradients expressed at locations relative to keypoint orientation θ
 - We could also have just rotated the whole image by $-\theta$, but that would be slower



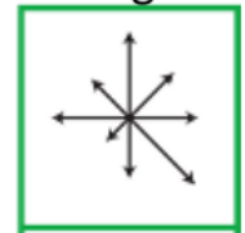
SIFT: an image region descriptor

Keypoint neighborhood



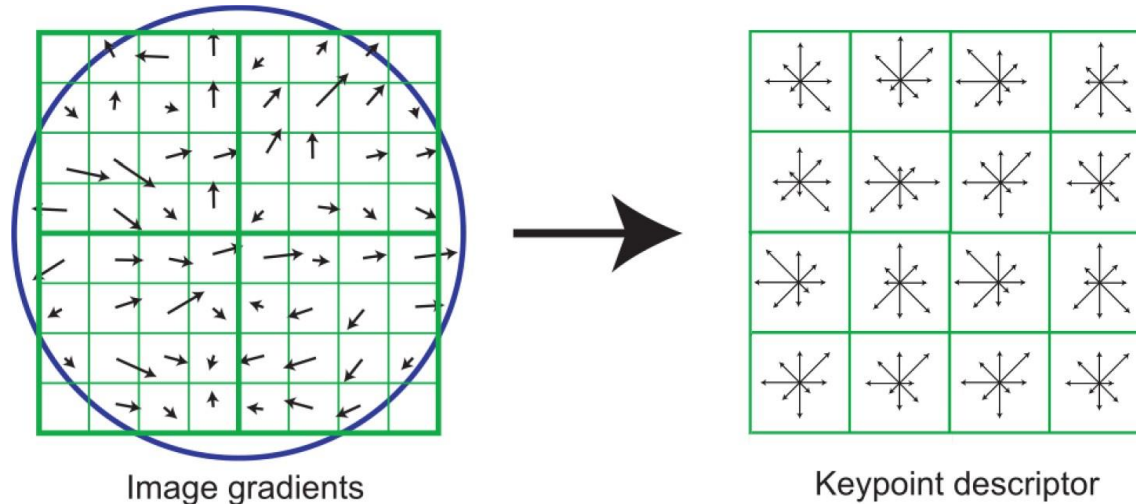
angle histogram

Orientation Histogram



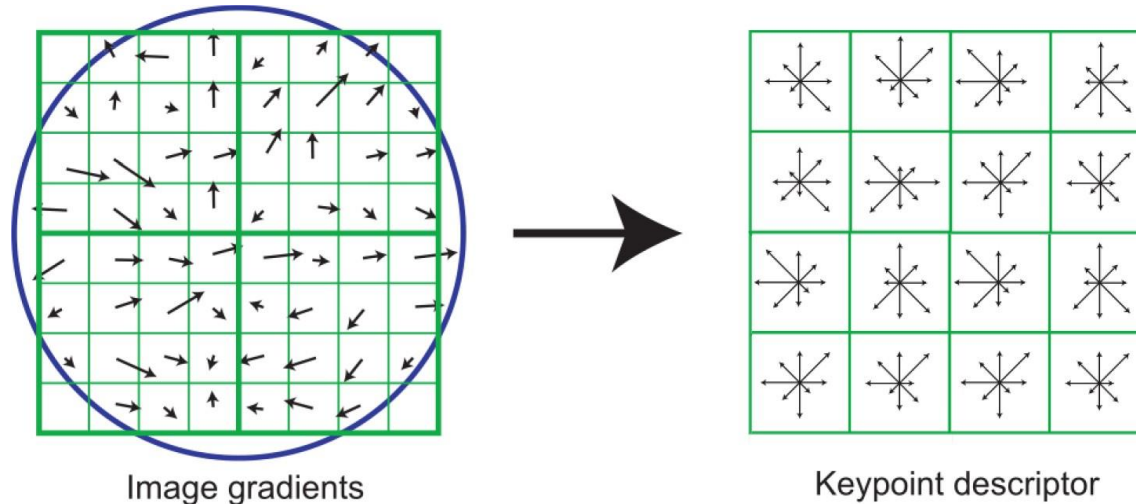
- Stack all gradients into a single vector?
- Using precise gradient locations is fragile. We'd like to allow some "slop" in the pixel configurations within the image, but still produce a very similar descriptor
- Create array of orientation histograms (a 1x1 array is shown), with 8 orientation bins

SIFT: an image region descriptor



- Create array of orientation histograms (a 4x4 array is shown)
- Put the rotated gradients into their local orientation histograms
 - A gradient's contribution is divided among the nearby histograms based on distance. If it's halfway between two histogram locations, it gives a half contribution to both.
 - Also, scale down gradient contributions for gradients far from the center and determine orientation weight by gradient magnitude
- The SIFT authors found that best results were with **8 orientation bins** per histogram, and a **4x4 histogram array**.

SIFT: an image region descriptor



- 8 orientation bins per histogram, and a 4x4 histogram array, yields $8 \times 4 \times 4 = 128$ numbers
- So a SIFT descriptor is a length 128 vector, which is invariant to rotation (because we rotated the gradient) and scale (because we worked with the scaled image from DoG)
- We can compare each vector from image A to each vector from image B to find matching keypoints!
 - Euclidean “distance” between descriptor vectors gives a good measure of keypoint similarity

SIFT: an image region descriptor

- Adding robustness to illumination changes:
 - Remember that the descriptor is made of gradients (differences between pixels), so it's already invariant to changes in brightness (e.g. adding 10 to all image pixels yields the exact same descriptor)
 - A higher-contrast (ratio of the brightest the darkest) photo will increase the magnitude of gradients linearly. So, to correct for contrast changes, normalize the vector (scale to length 1.0)
 - Very large image gradients are usually from unreliable 3D illumination effects (glare, etc). So, to reduce their effect, clamp all values in the vector to be ≤ 0.2 (an experimentally tuned value). Then normalize the vector again.
 - Result is a vector which is fairly invariant to illumination changes.

SIFT: an image region descriptor

- Sensitivity to number of histogram orientations and array size

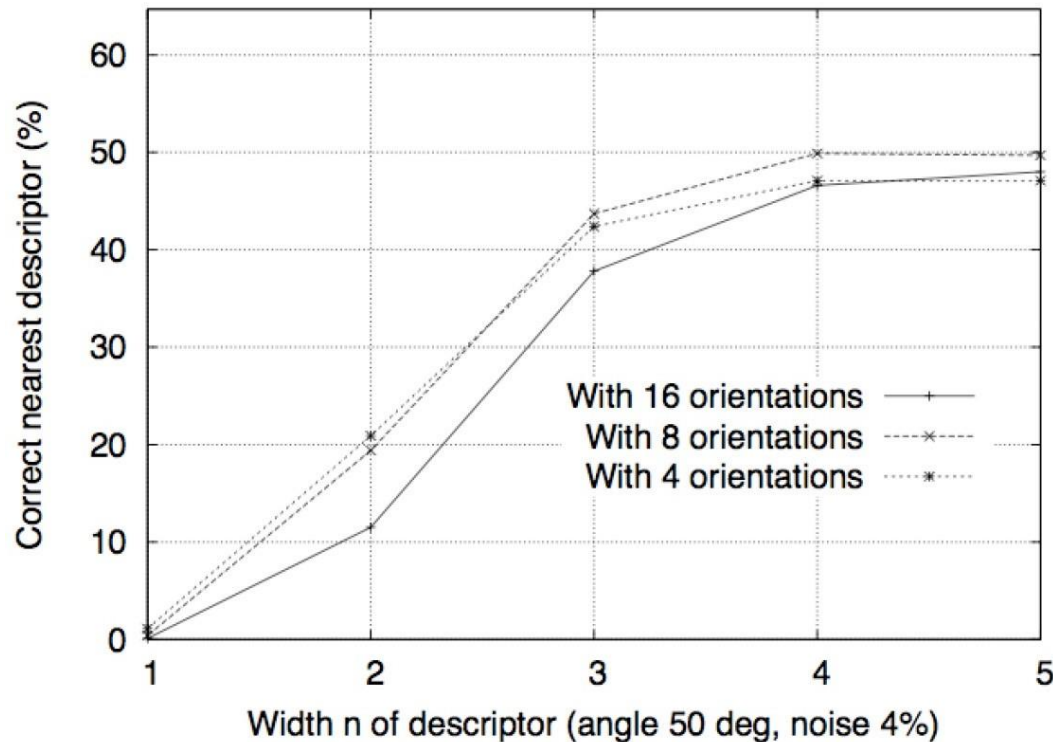
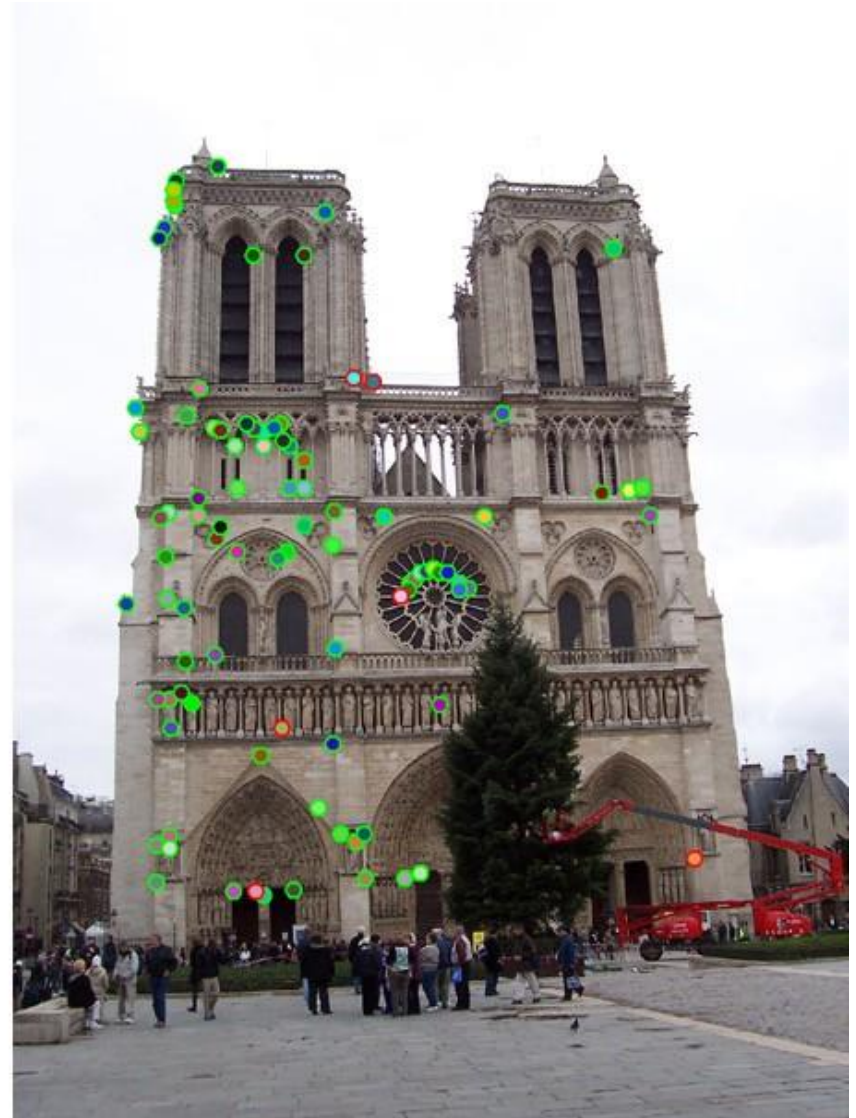
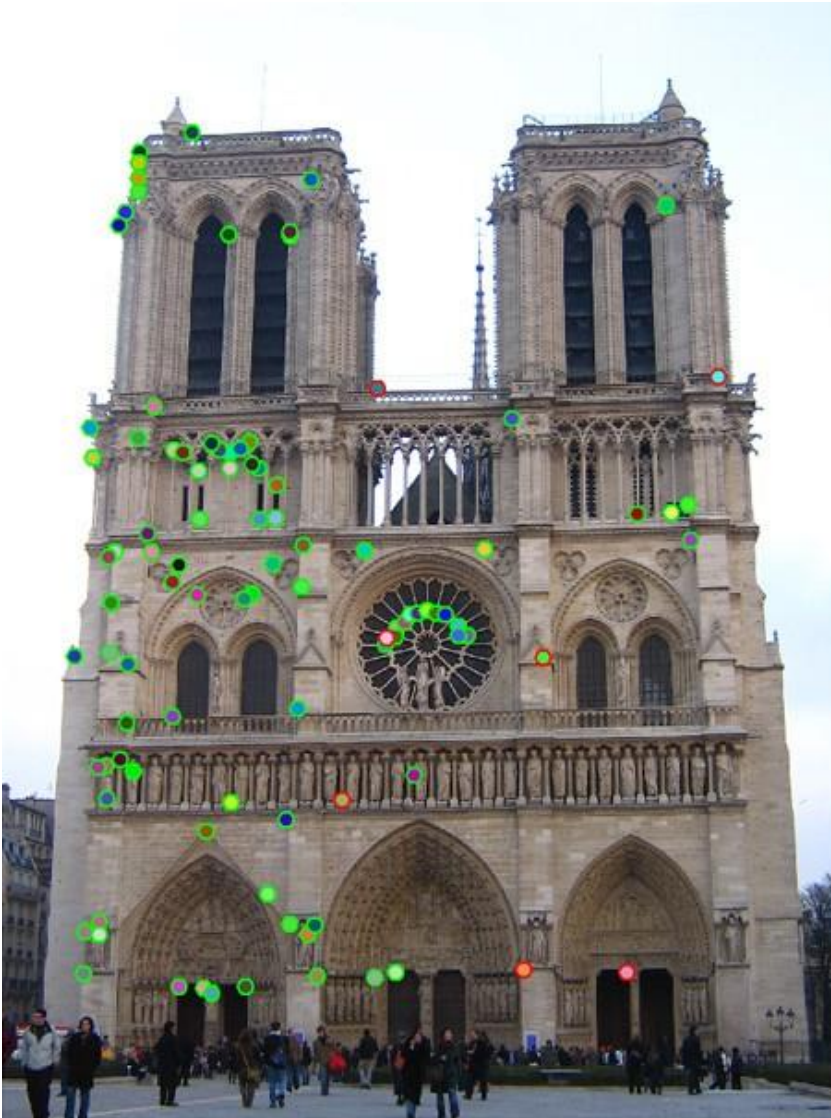


Figure 8: This graph shows the percent of keypoints giving the correct match to a database of 40,000 keypoints as a function of width of the $n \times n$ keypoint descriptor and the number of orientations in each histogram. The graph is computed for images with affine viewpoint change of 50 degrees and addition of 4% noise.

Which features match?





Feature matching

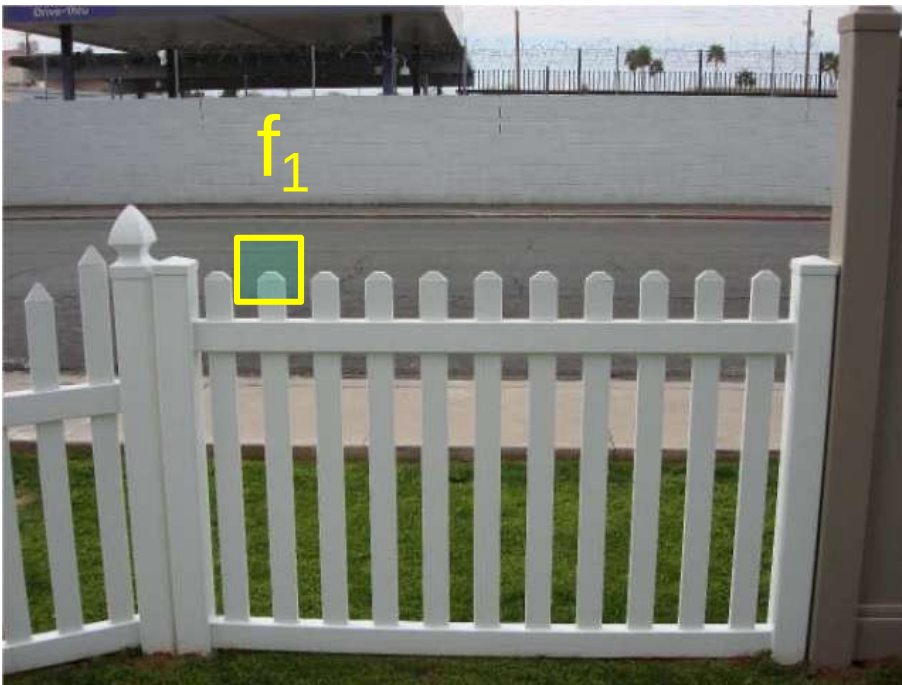
Given a feature in I_1 , how to find the best match in I_2 ?

1. Define distance function that compares two descriptors
2. Test all the features in I_2 , find the one with min distance
(can be accelerated with a nearest neighbors search data structure, like a kd-tree)

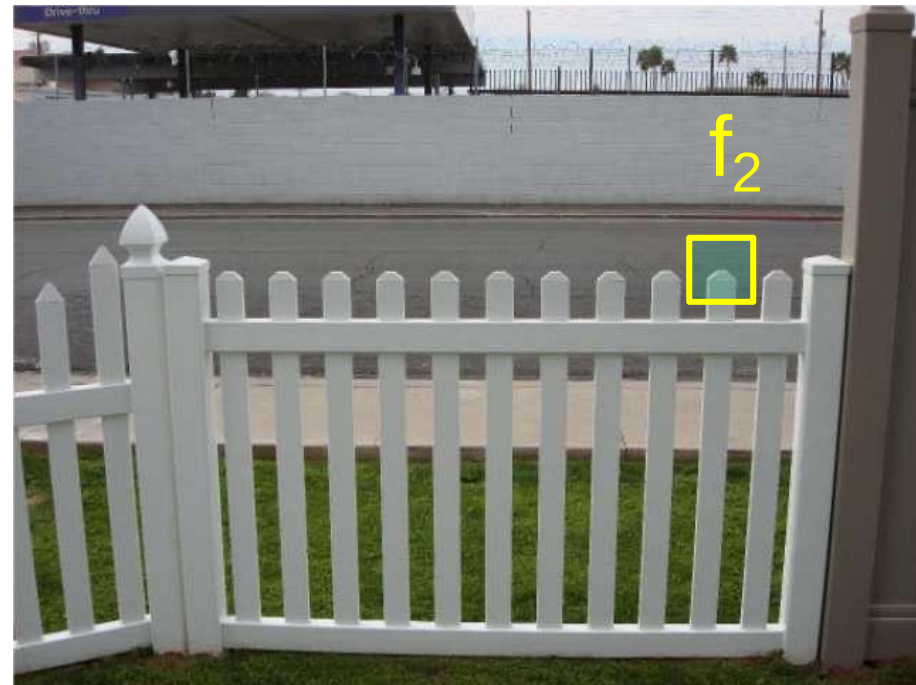
Feature distance

How to define the difference between two features f_1, f_2 ?

- Simple approach: L_2 distance, $||f_1 - f_2||$
- can give small distances for ambiguous (incorrect) matches



I_1

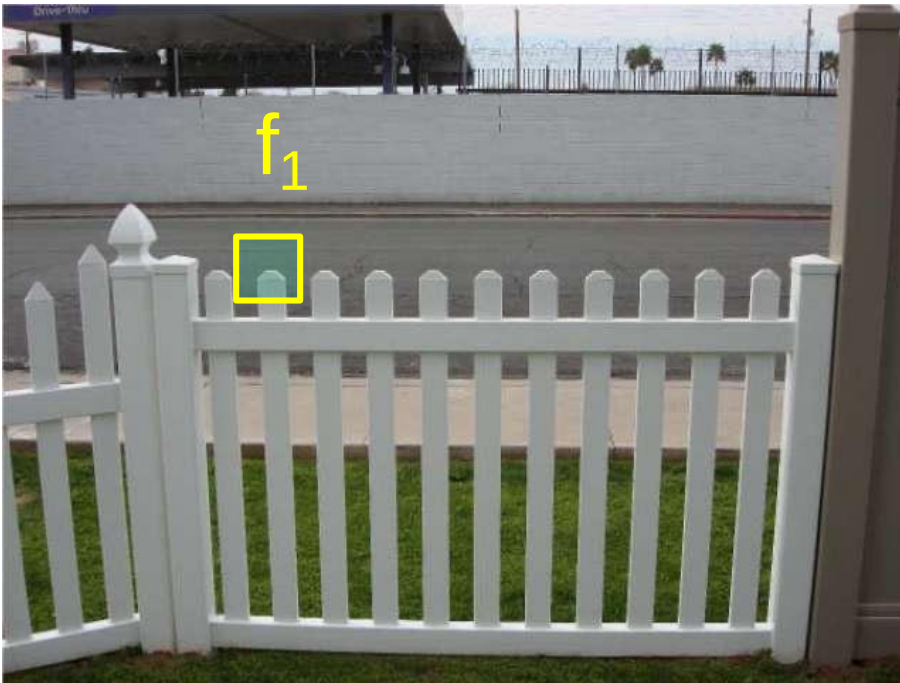


I_2

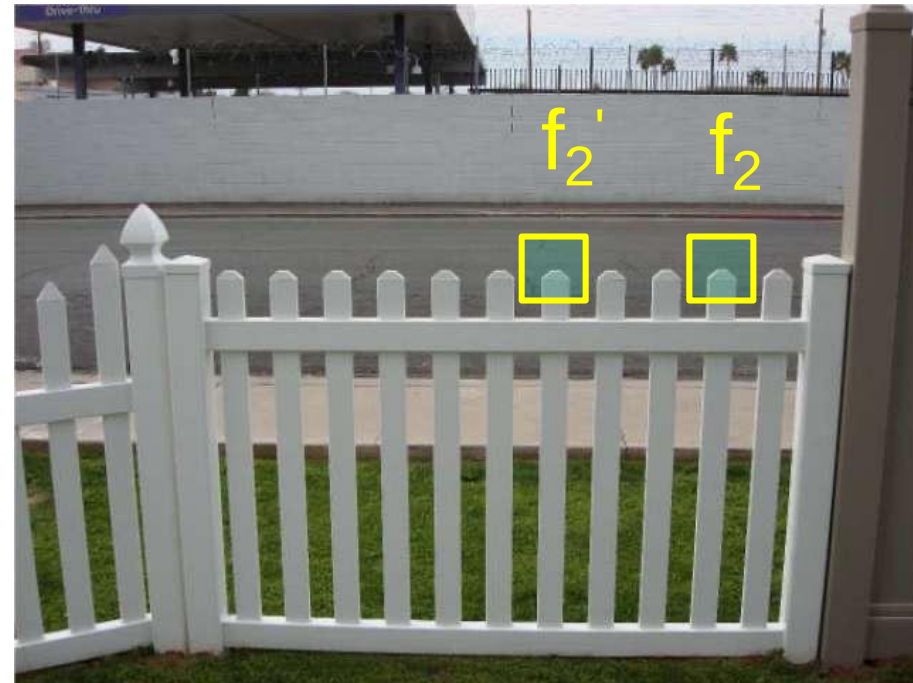
Feature distance

How to define the difference between two features f_1, f_2 ?

- Better approach: ratio distance = $||f_1 - f_2|| / ||f_1 - f_2'||||$
 - f_2 is best match to f_1 in I_2
 - f_2' is 2nd best match to f_1 in I_2
 - gives large values for ambiguous matches



I_1

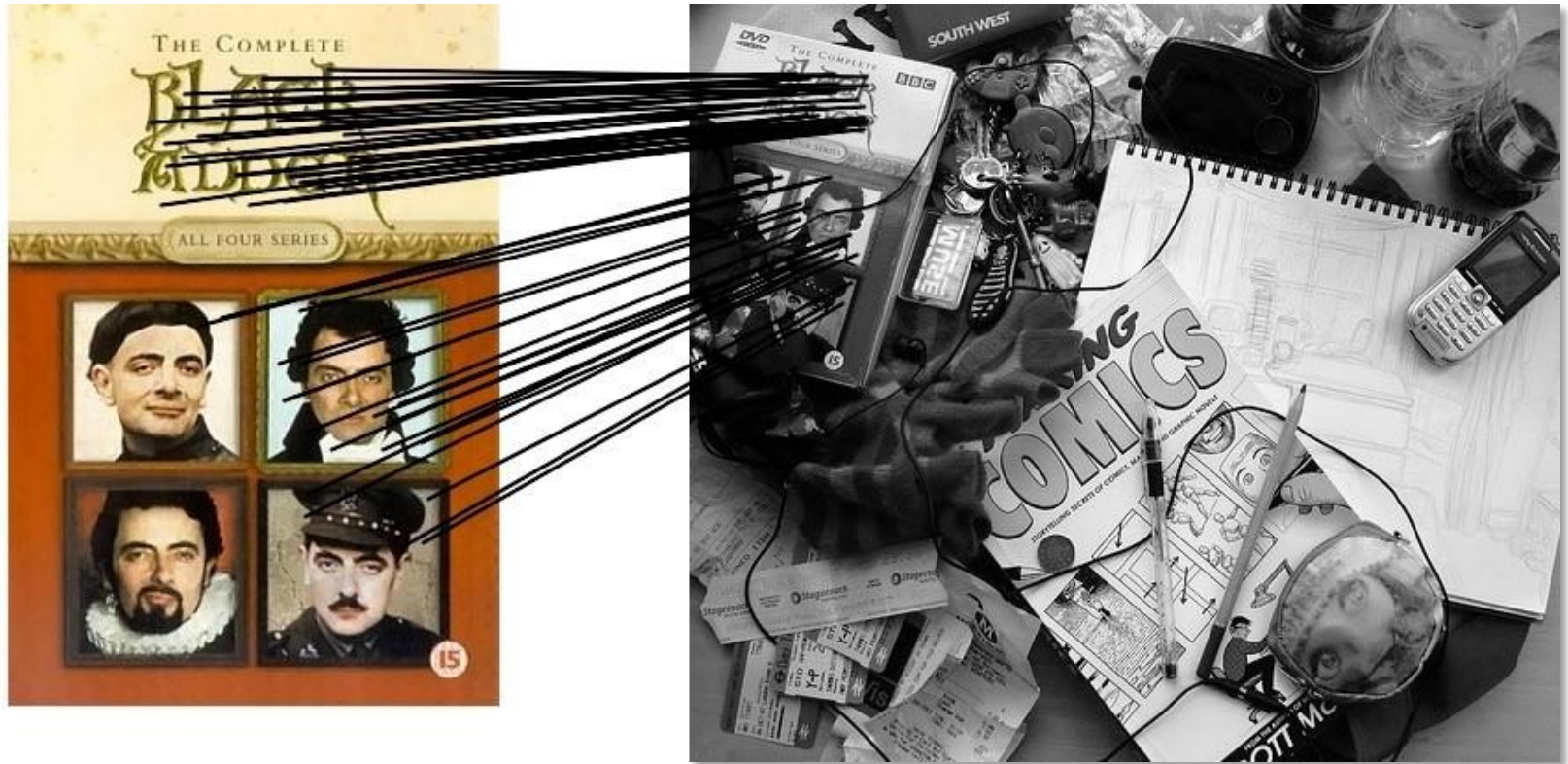


I_2

Feature distance

- Does the L_2 distance vs “ratio distance” change the best match to a given feature in image 1?
- No, but it changes the distance, and it can change the ordering of matches from good to bad
- After we compute a set of matches, we threshold by distance (that is, throw out matches with distance $>$ threshold)

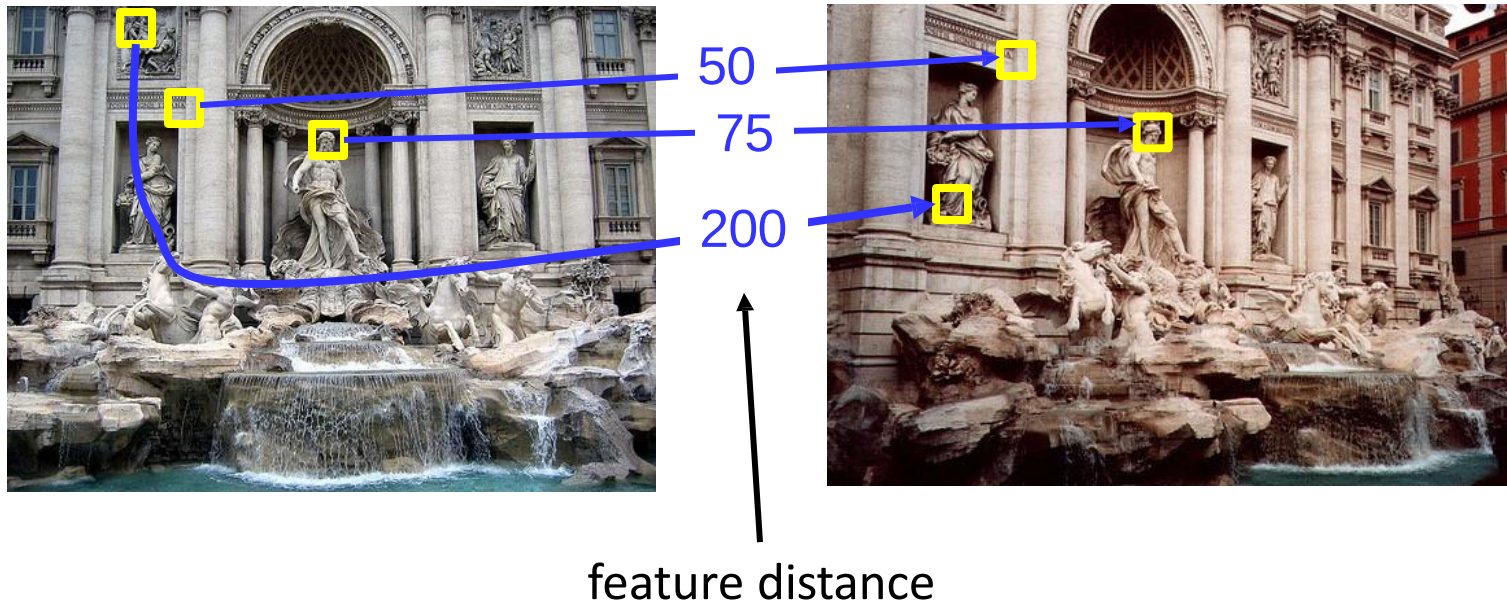
Feature matching example



58 matches (thresholded by ratio score)

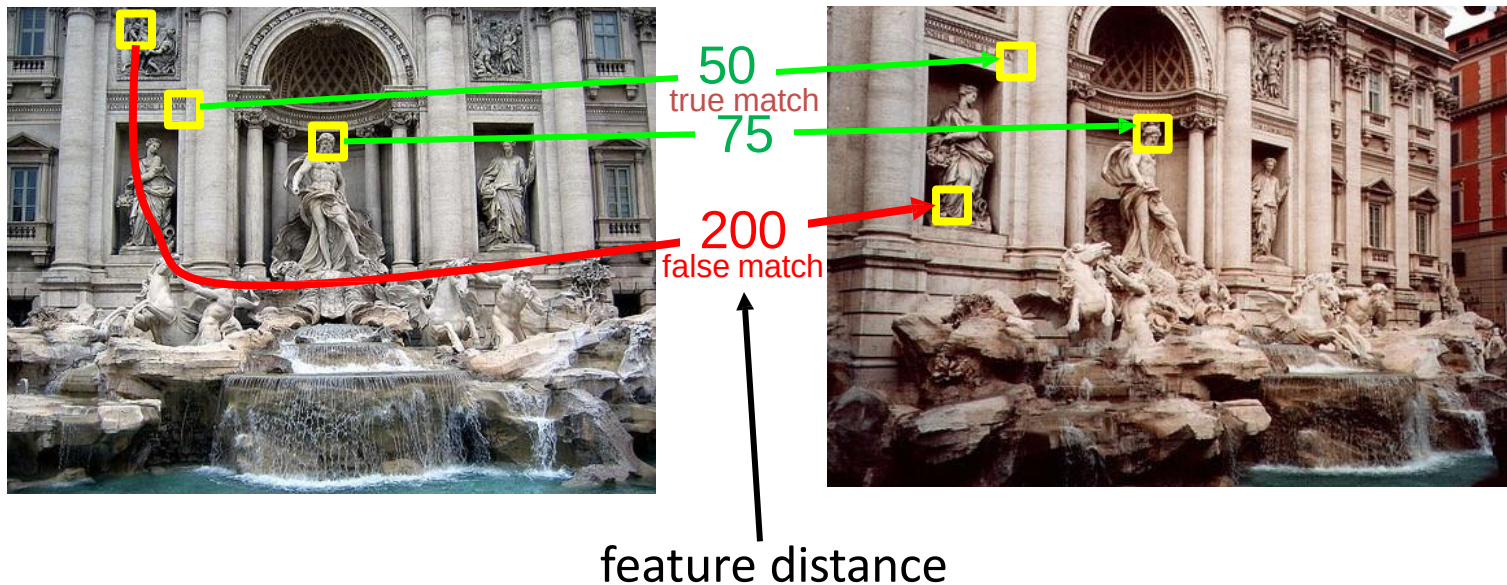
Evaluating the results

How can we measure the performance of a feature matcher?



True/false positives

How can we measure the performance of a feature matcher?



The distance threshold affects performance

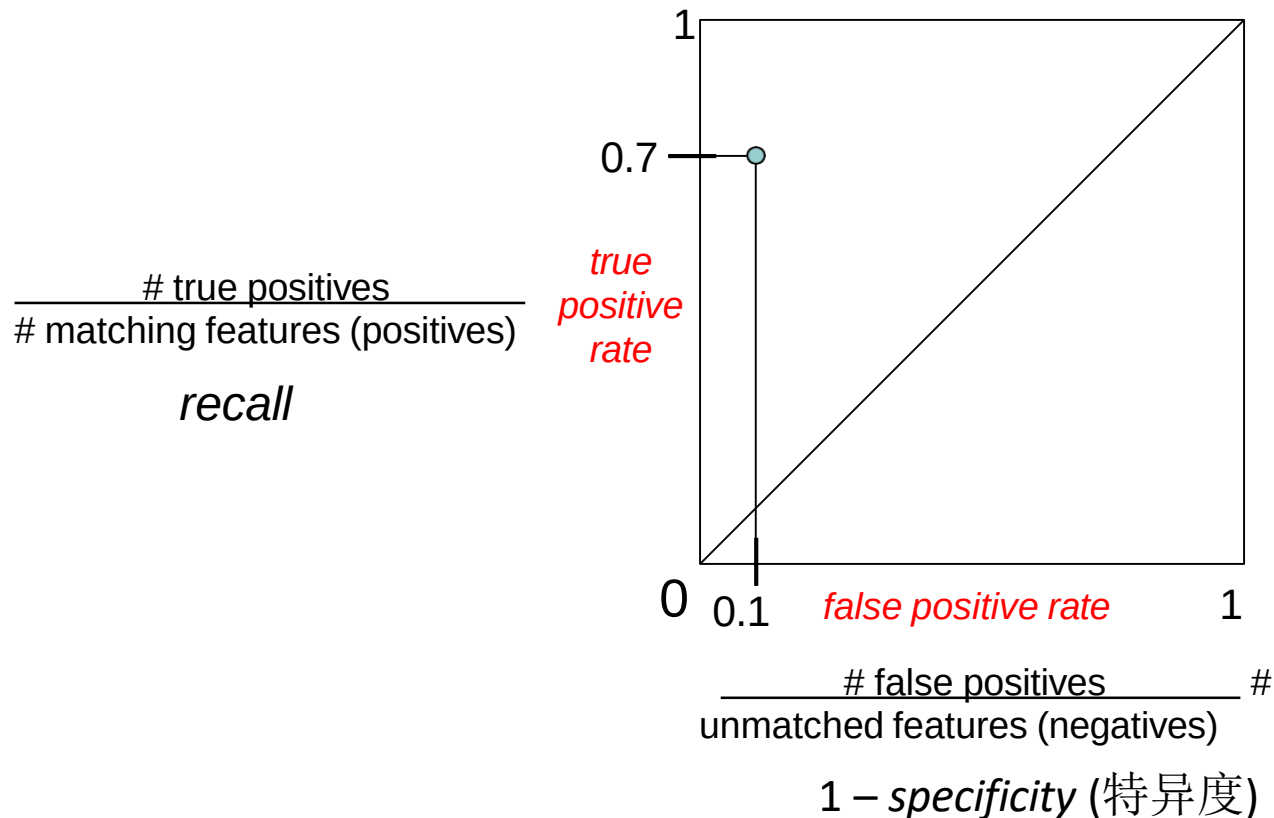
- True positives = # of detected matches that are correct
 - Suppose we want to maximize these—how to choose threshold?
- False positives = # of detected matches that are incorrect
 - Suppose we want to minimize these—how to choose threshold?

Example

- Suppose our matcher computes 1,000 matches between two images
 - 800 are correct matches, 200 are incorrect (according to an oracle that gives us ground truth matches)
 - A given threshold (e.g., ratio distance = 0.6) gives us 600 correct matches and 100 incorrect matches that survive the threshold
 - True positive rate = $600 / 800 = \frac{3}{4}$
 - False positive rate = $100 / 200 = \frac{1}{2}$

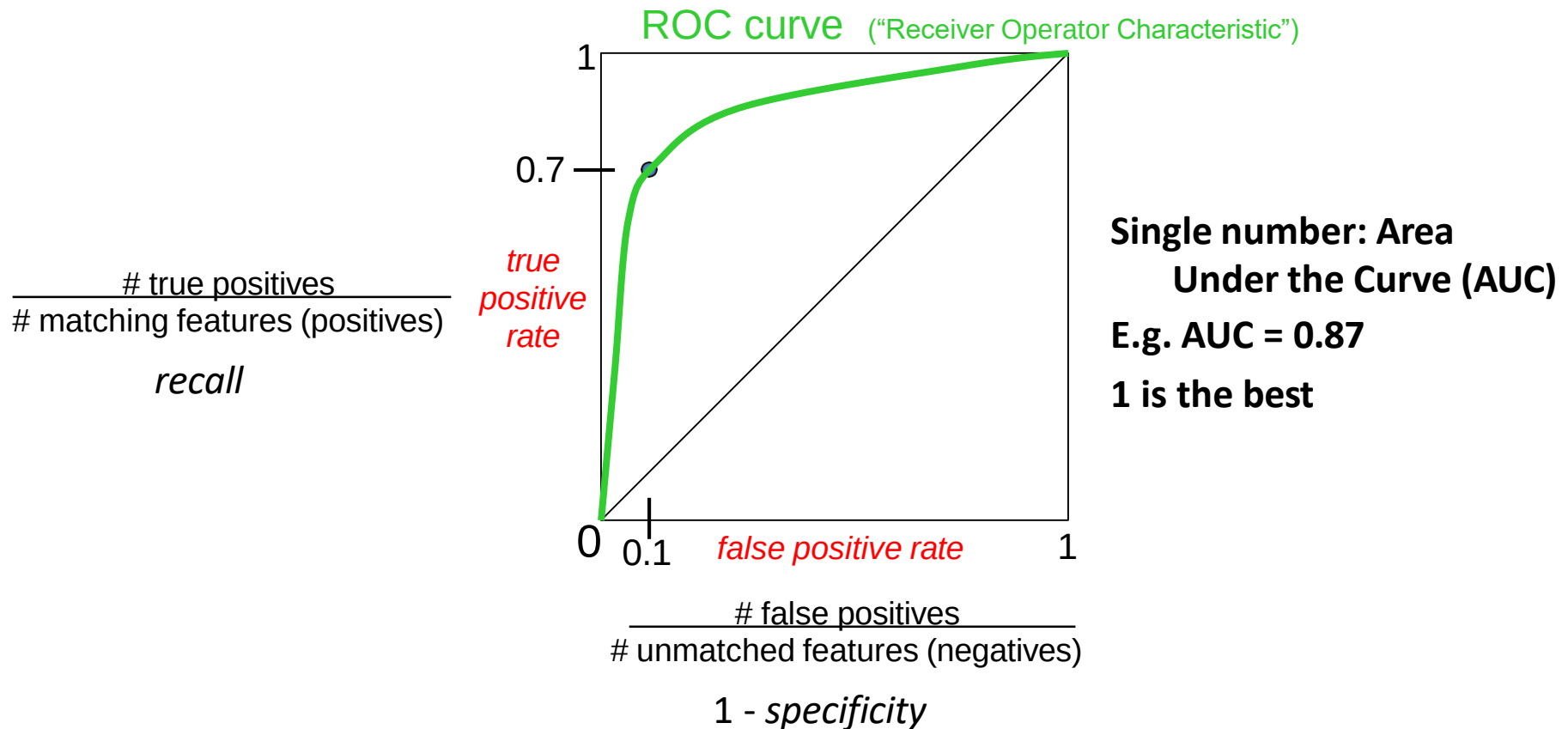
Evaluating the results

How can we measure the performance of a feature matcher?



Evaluating the results

How can we measure the performance of a feature matcher?



ROC curves – summary

- By thresholding the match distances at different thresholds, we can generate sets of matches with different true/false positive rates
- ROC curve is generated by computing rates at a set of threshold values swept through the full range of possible thresholds
- Area under the ROC curve (AUC) summarizes the performance of a feature pipeline (higher AUC is better)

SIFT: an image region descriptor

- Recognition of specific objects, scenes



Schmid and Mohr 1997



Sivic and Zisserman, 2003



Rothganger et al. 2003



Lowe 2002

SIFT: an image region descriptor

- Applications of local invariant features
 - Wide baseline stereo
 - Motion tracking
 - Panoramas
 - Mobile robot navigation
 - 3D reconstruction
 - Recognition
 - ...



Summary

- SIFT Descriptor
 - Descriptor calculation
 - Experimental Analysis
 - Applications

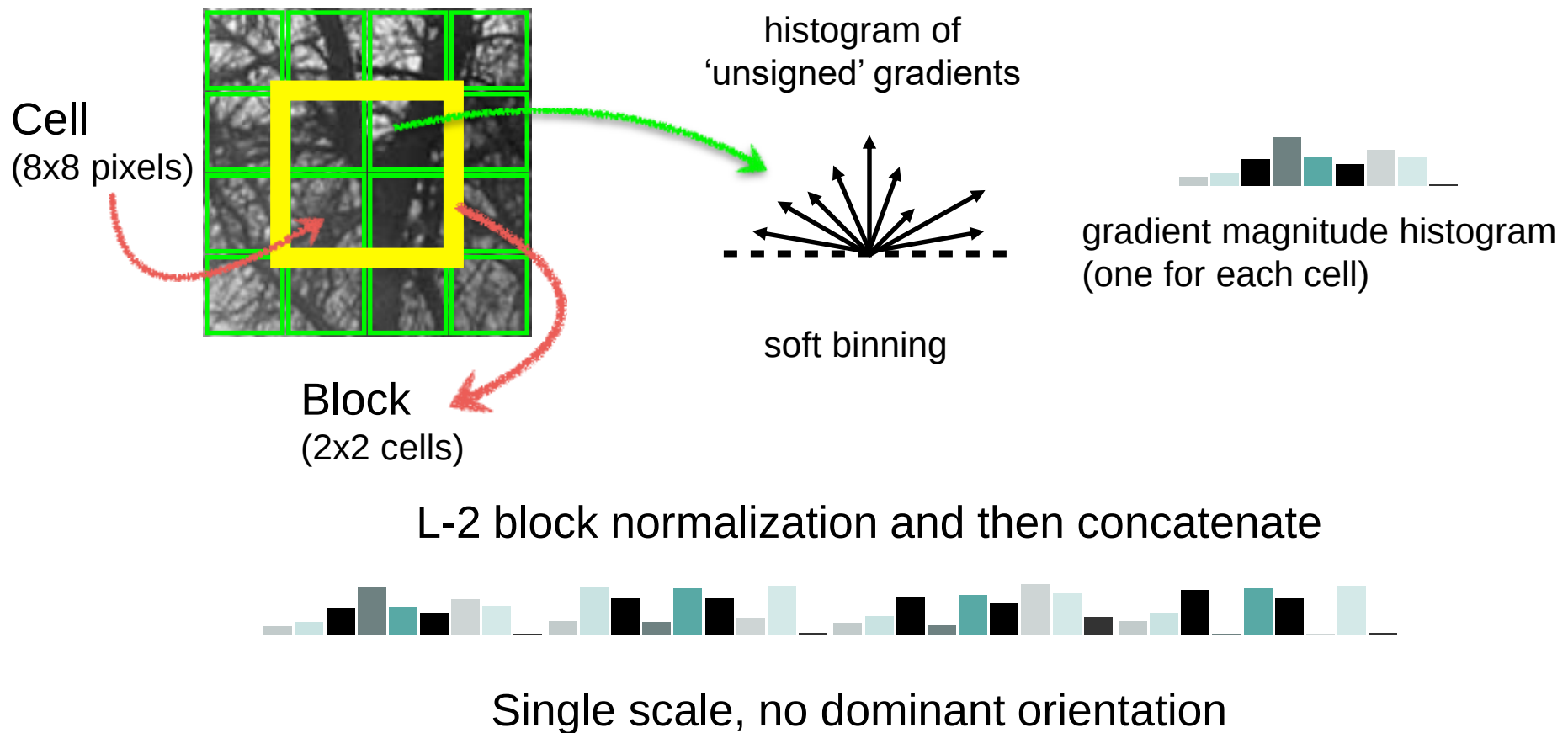


What we will learn today?

- Scale invariant keypoint detection
 - Automatic scale selection
 - Difference-of-Gaussian (DoG) detector
- SIFT: an image region descriptor
- HoG: another image region descriptor
- Application: Panorama

HOG

Dalal, Triggs. **Histograms of Oriented Gradients** for Human Detection. CVPR, 2005



Pedestrian detection

1 cell step size

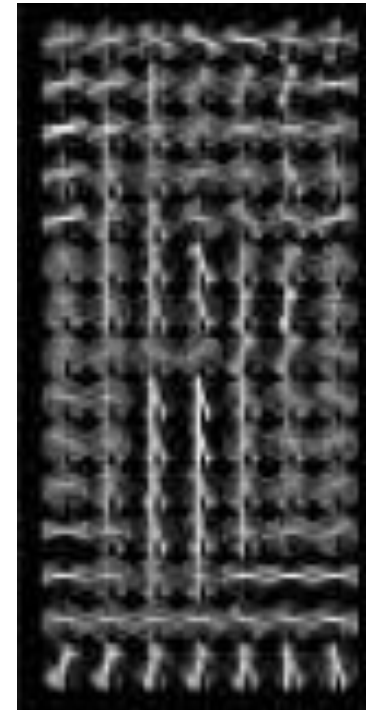


128 pixels
16 cells
15 blocks



visualization

$$15 \times 7 \times 4 \times 9 = 3780$$



64 pixels
8 cells
7 blocks



Redundant representation due to overlapping blocks

How many times is each inner cell encoded?

<http://chrisjmccormick.wordpress.com/2013/05/09/hog-person-detector-tutorial/>

What we will learn today?

- Scale invariant region detection
 - Automatic scale selection
 - Difference-of-Gaussian (DoG) detector
- SIFT: an image region descriptor
- HoG: another image region descriptor
- **Application: Panorama**

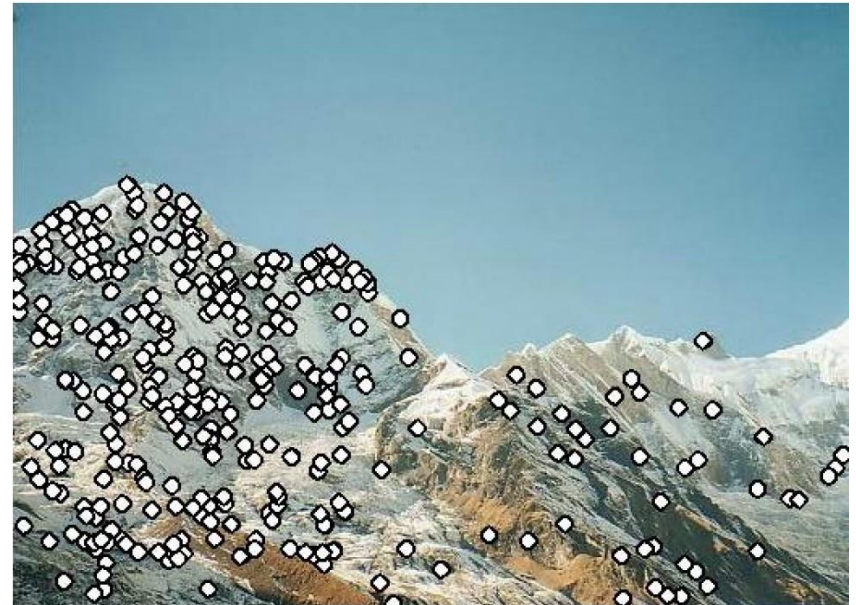
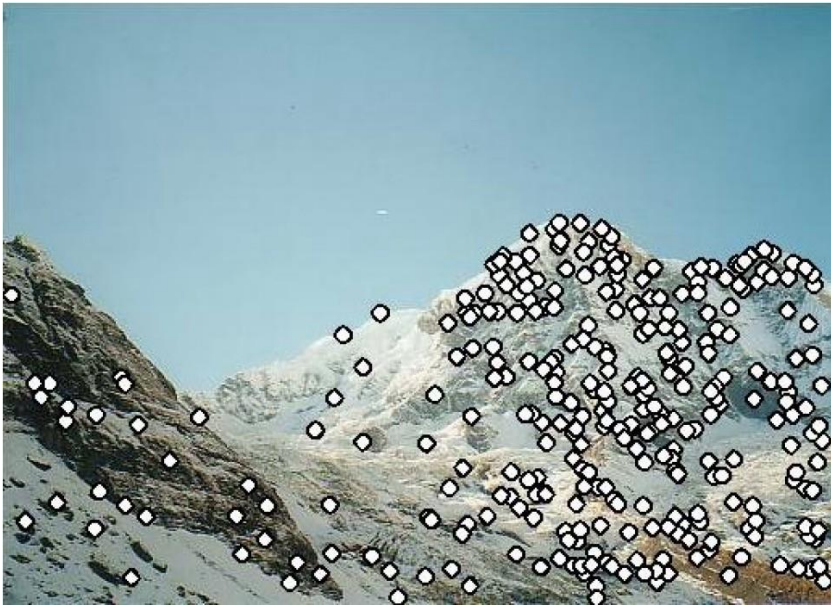
Application: Panorama

- Application: Image Stitching



Application: Panorama

- Application: Image Stitching

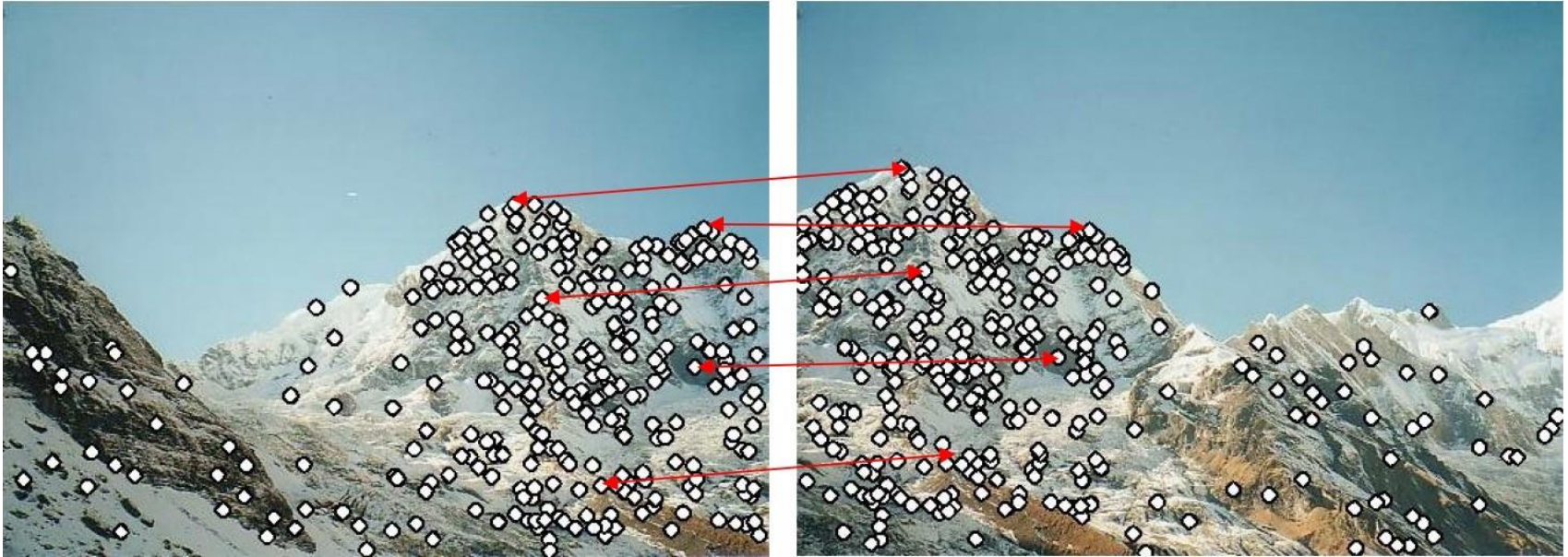


Procedure:

- Detect feature points in both images

Application: Panorama

- Application: Image Stitching



Procedure:

- Detect feature points in both images
- Find corresponding pairs

Application: Panorama

- Application: Image Stitching

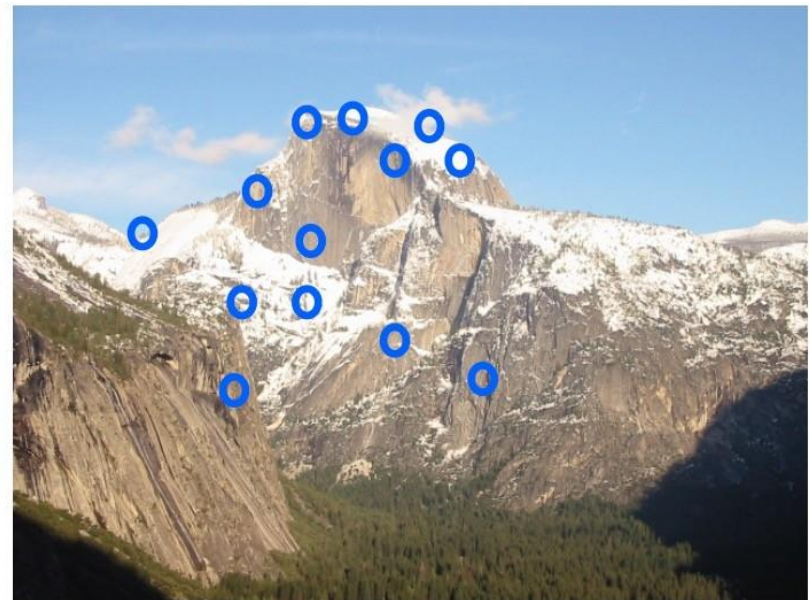
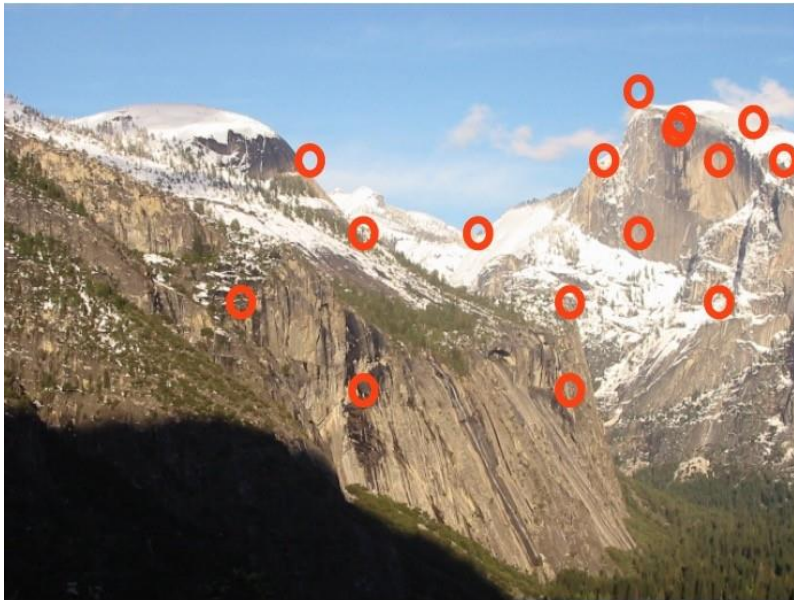


Procedure:

- Detect feature points in both images
- Find corresponding pairs
- Use these pairs to align the images

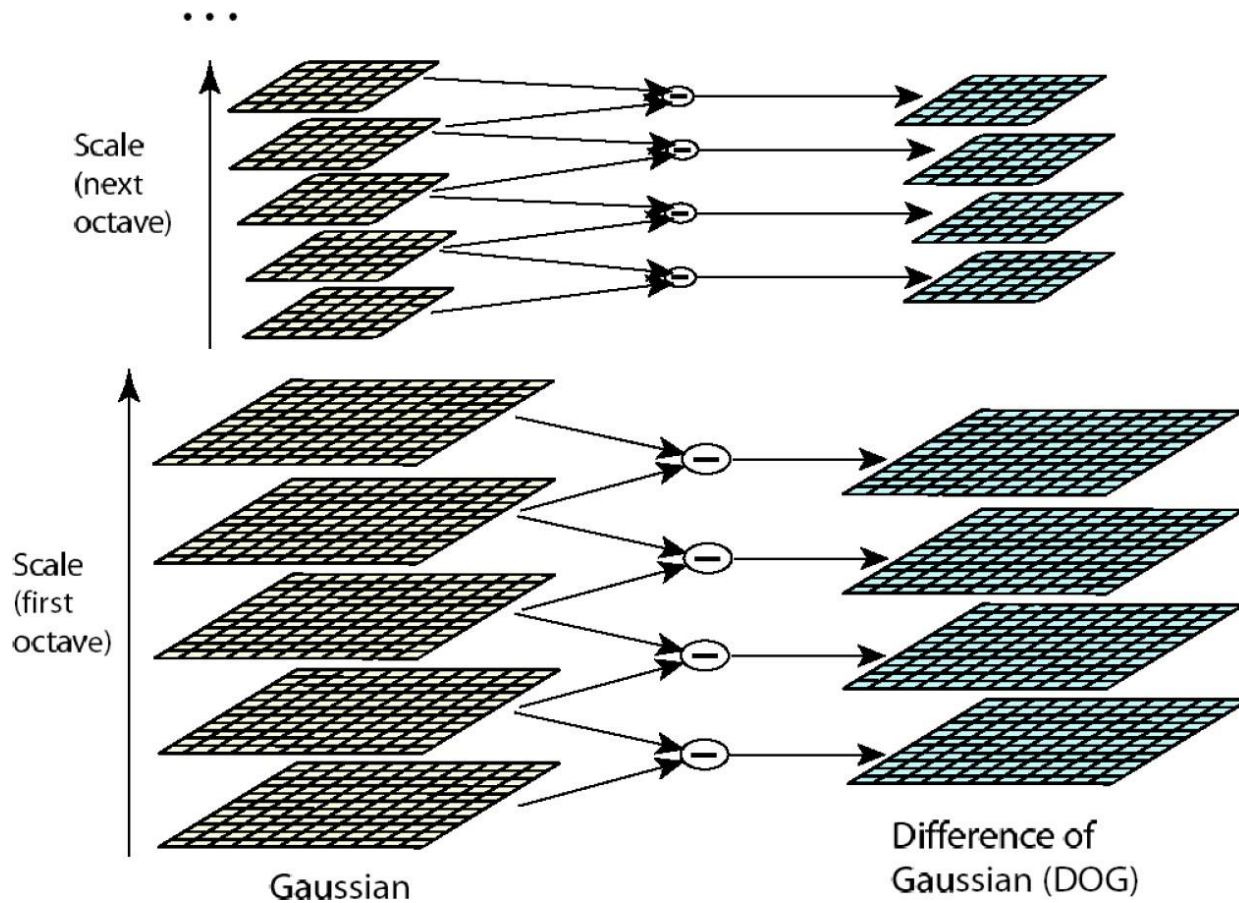
Application: Panorama

- Main Flow
 - Detect key points



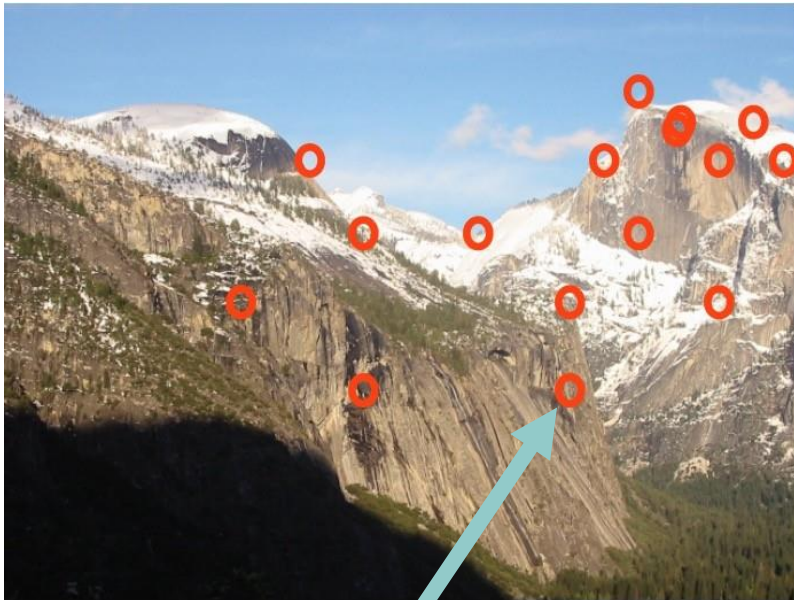
Application: Panorama

- Detect Key Points

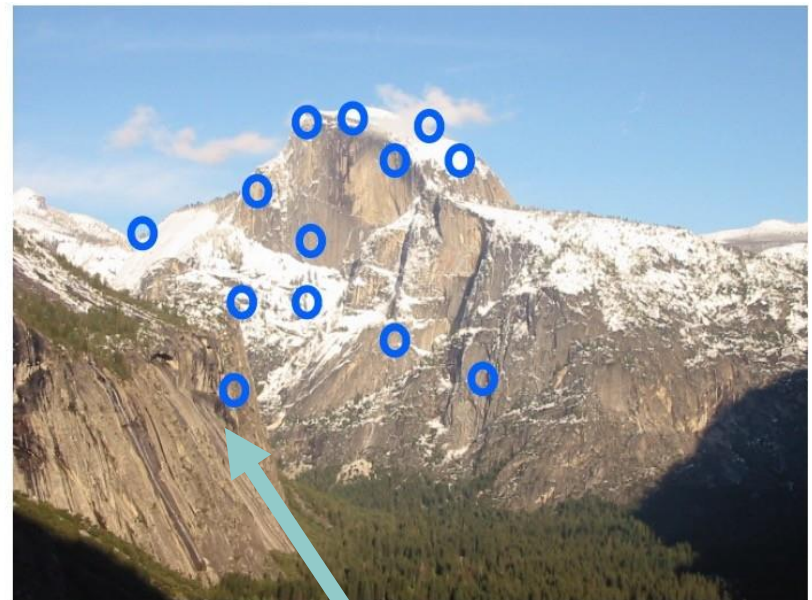


Application: Panorama

- Main Flow
 - Detect key points
 - Build the SIFT descriptors



$(u_1, u_2, \dots, u_{128})$



$(v_1, v_2, \dots, v_{128})$

Application: Panorama

- Build the SIFT Descriptors

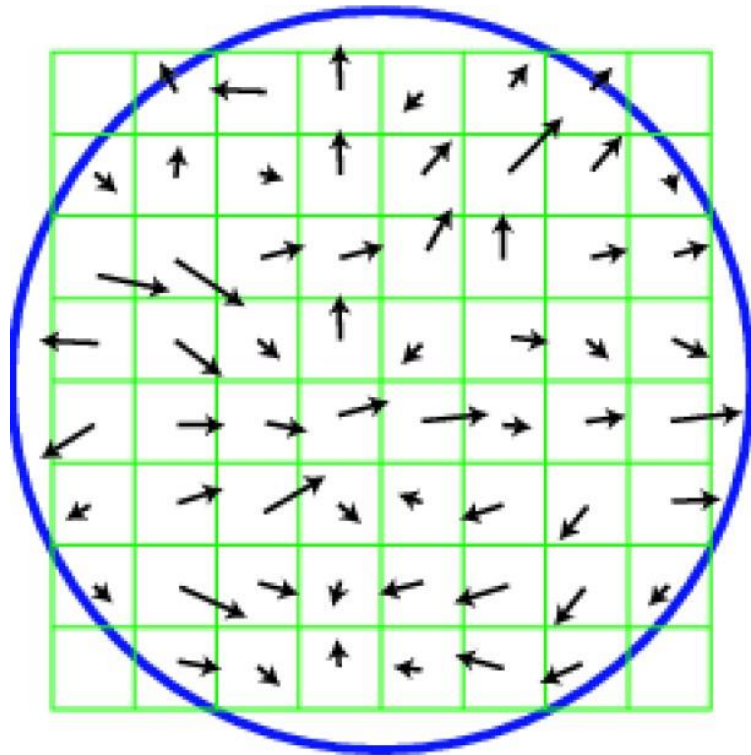
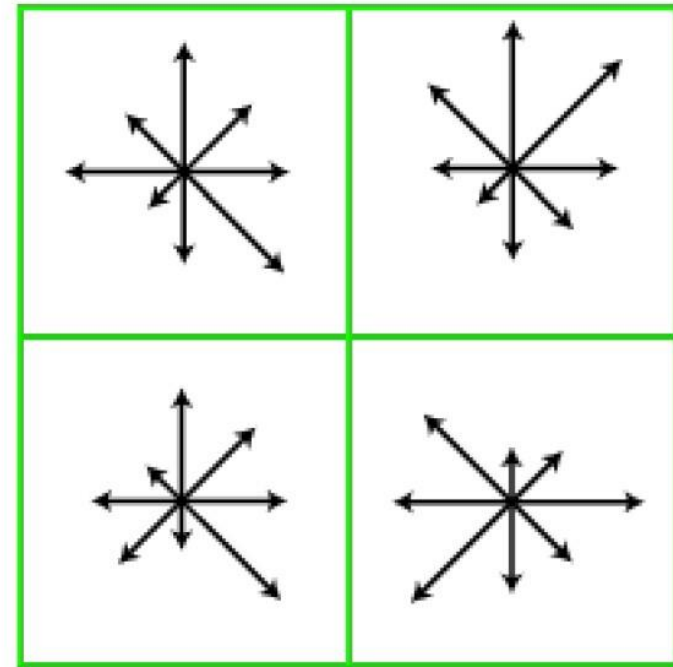
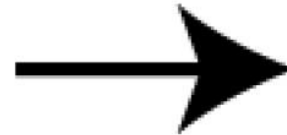


Image gradients



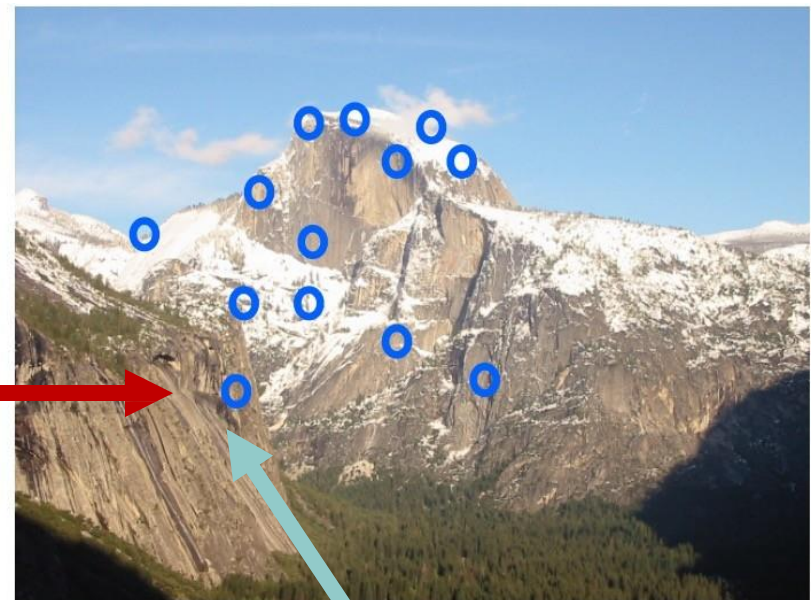
Keypoint descriptor

Application: Panorama

- Main Flow
 - Detect key points
 - Build the SIFT descriptors
 - Match SIFT descriptors



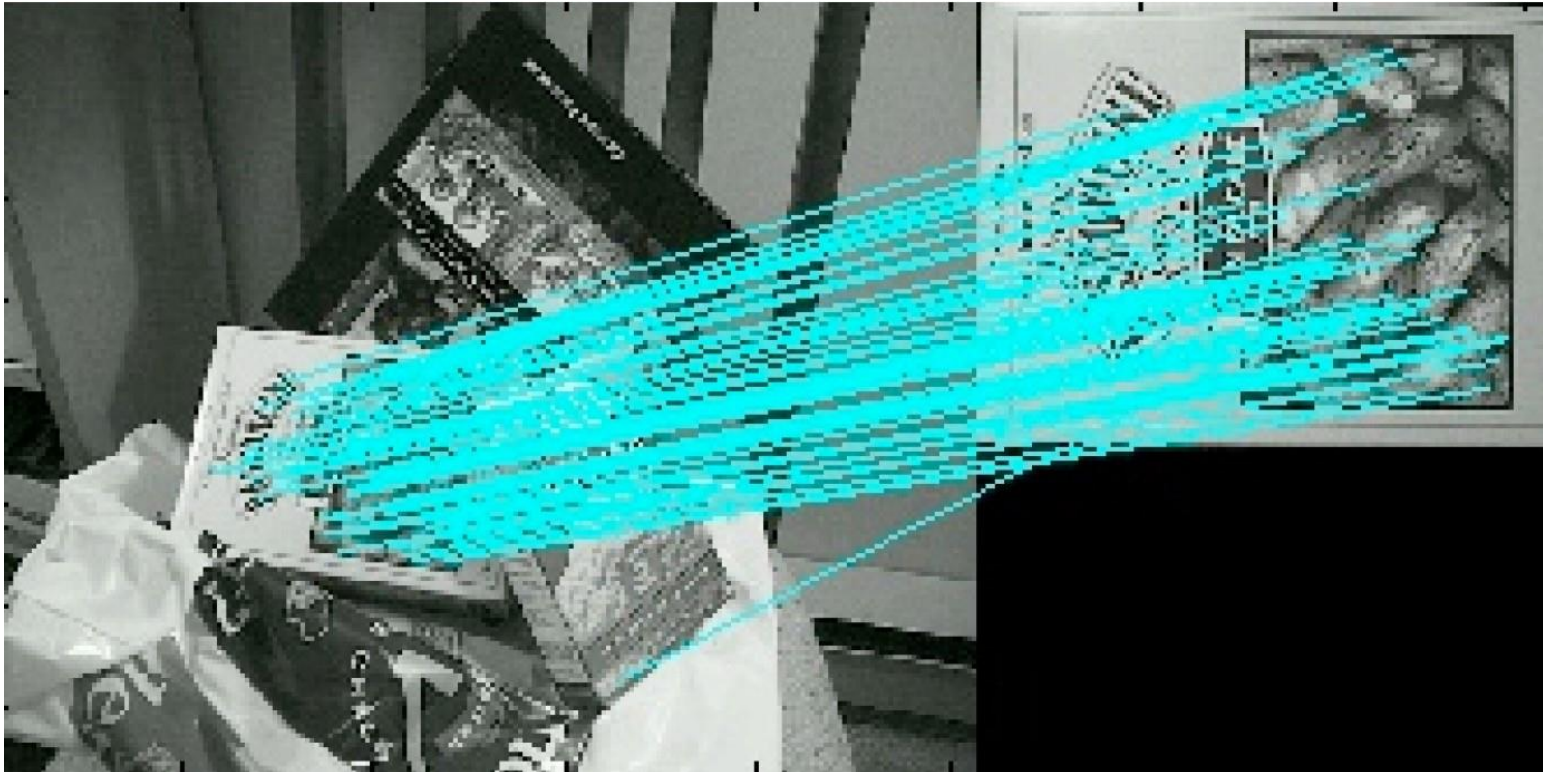
$(u_1, u_2, \dots, u_{128})$



$(v_1, v_2, \dots, v_{128})$

Application: Panorama

- Match SIFT Descriptors
 - Euclidean distance between descriptors



Application: Panorama

- Skeleton Code
 - Match SIFT descriptors (6 lines of code)
 - Input: D1, D2, thresh (default 0.7)
 - Output: match [D1's index, D2's index]
 - Try to use one for loop

Application: Panorama

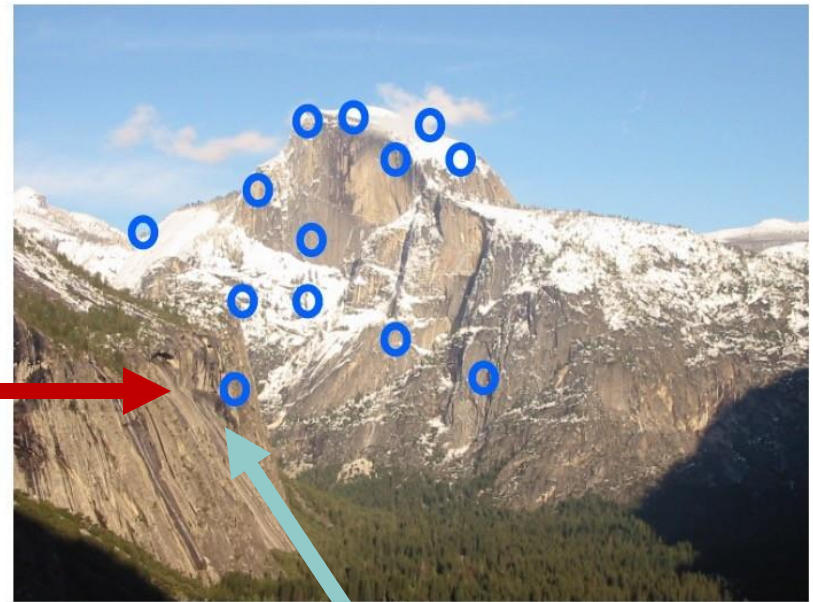
● Main Flow

- Detect key points
- Build the SIFT descriptors
- Match SIFT descriptors
- Fitting the transformation

$$T = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ 0 & 0 & 1 \end{bmatrix}$$



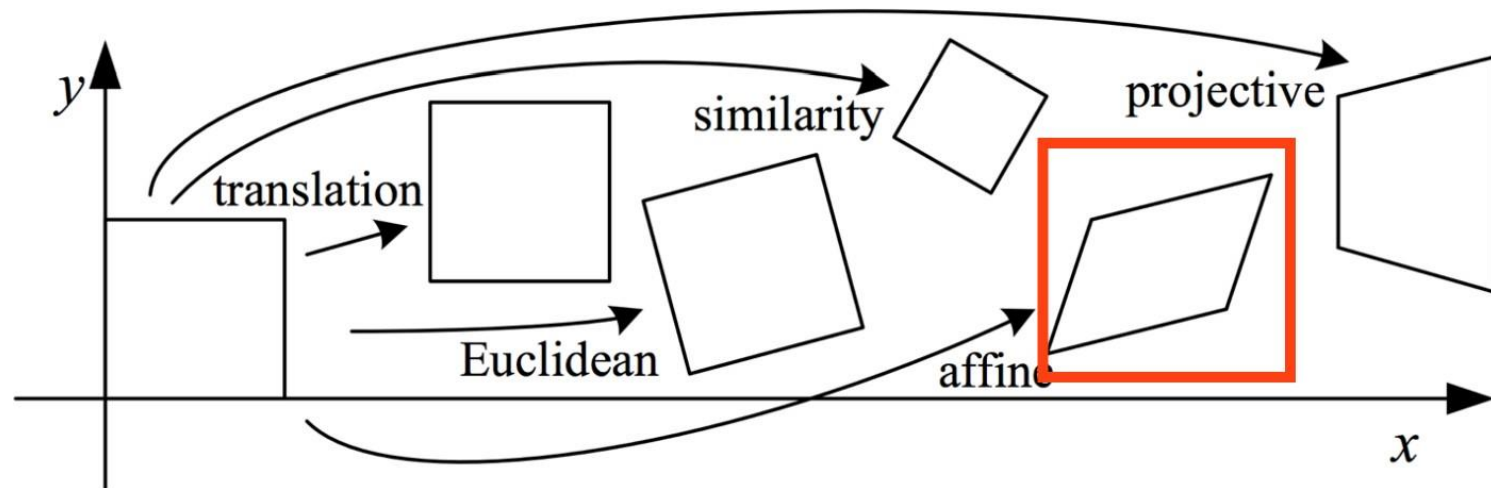
$(u_1, u_2, \dots, u_{128})$



$(v_1, v_2, \dots, v_{128})$

Application: Panorama

- Fitting the transformation
 - 2D transformations



Application: Panorama

- Skeleton Code
 - Fit the transformation matrix

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

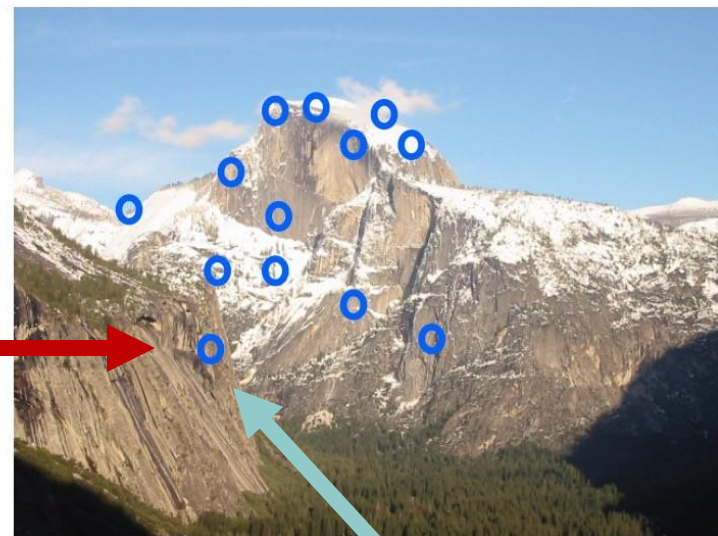
- Six variables
 - each point give two equations
 - at least three points
- Least squares

Application: Panorama

- Main Flow
 - Detect key points
 - Build the SIFT descriptors
 - Match SIFT descriptors
 - Fitting the transformation
 - RANSAC



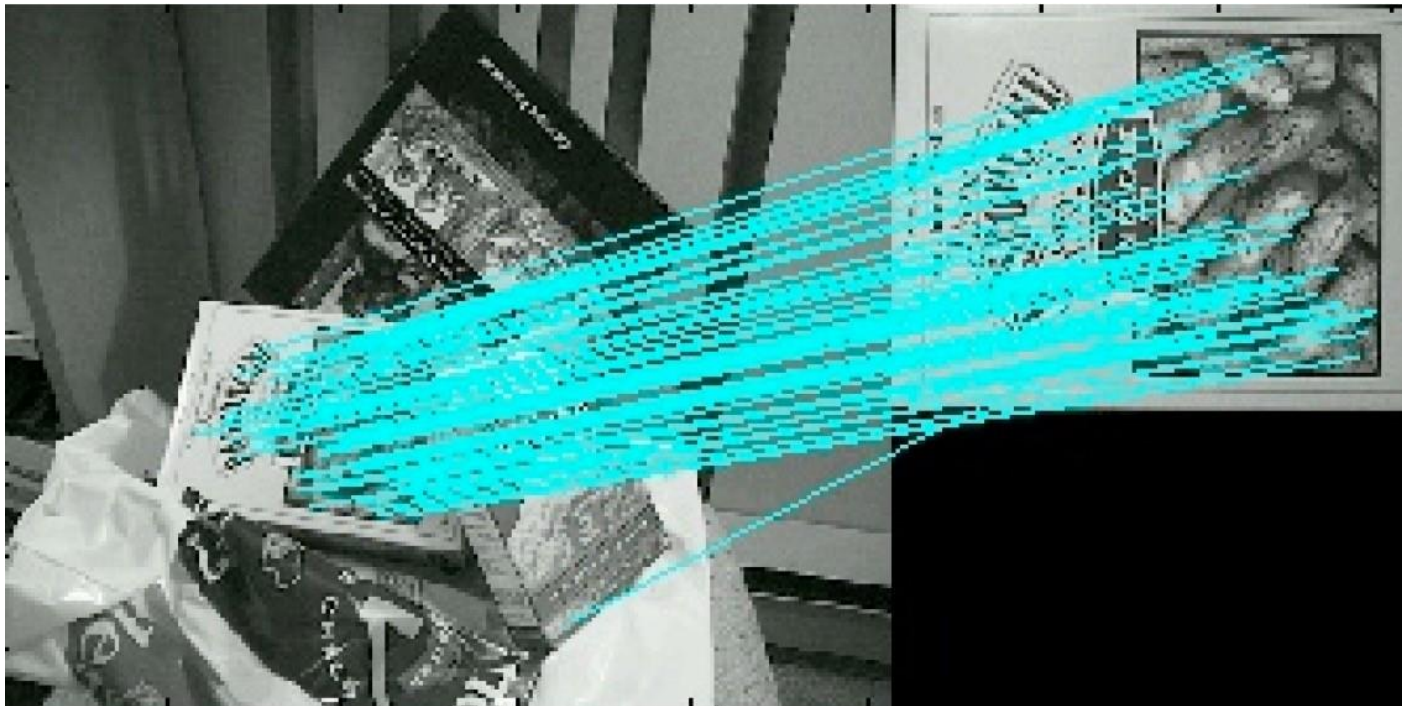
$(u_1, u_2, \dots, u_{128})$



$(v_1, v_2, \dots, v_{128})$

Application: Panorama

- RANSAC
 - A further refinement of matches



RANSAC: Algorithm

Algorithm 15.4: RANSAC: fitting lines using random sample consensus

Determine:

n — the smallest number of points required

k — the number of iterations required

t — the threshold used to identify a point that fits well

d — the number of nearby points required
to assert a model fits well

Until k iterations have occurred

Draw a sample of n points from the data
uniformly and at random

Fit to that set of n points

For each data point outside the sample

Test the distance from the point to the line
against t ; if the distance from the point to the line
is less than t , the point is close

end

If there are d or more points close to the line
then there is a good fit. Refit the line using all
these points.

end

Use the best fit from this collection, using the
fitting error as a criterion

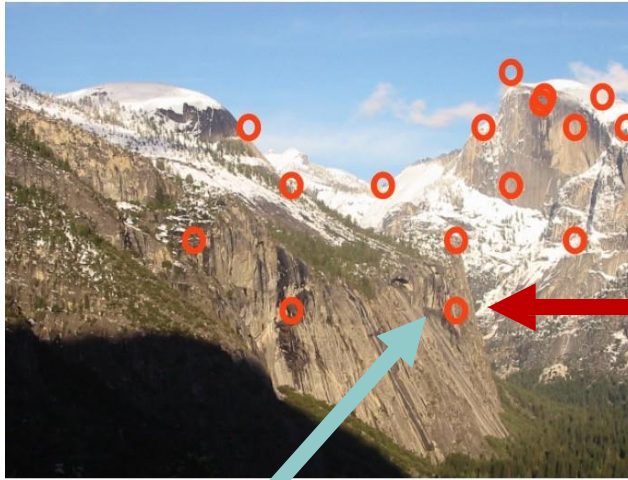
Application: Panorama

- Skeleton Code
 - RANSAC
 - Compute Error

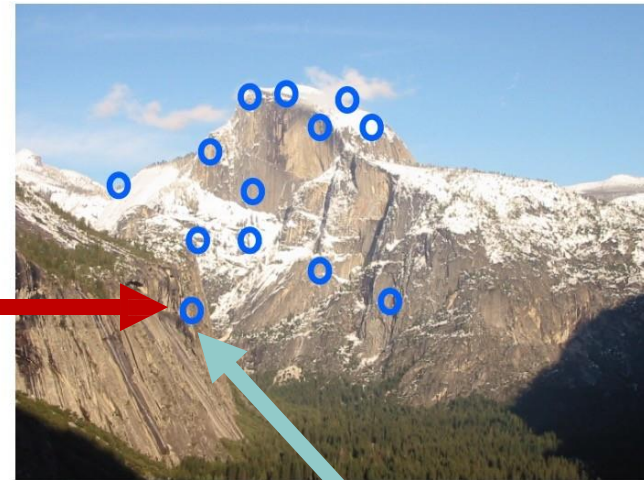
$$\left\| \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} - H \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \right\|_2$$

Application: Panorama

- Main Flow



$(u_1, u_2, \dots, u_{128})$



$(v_1, v_2, \dots, v_{128})$



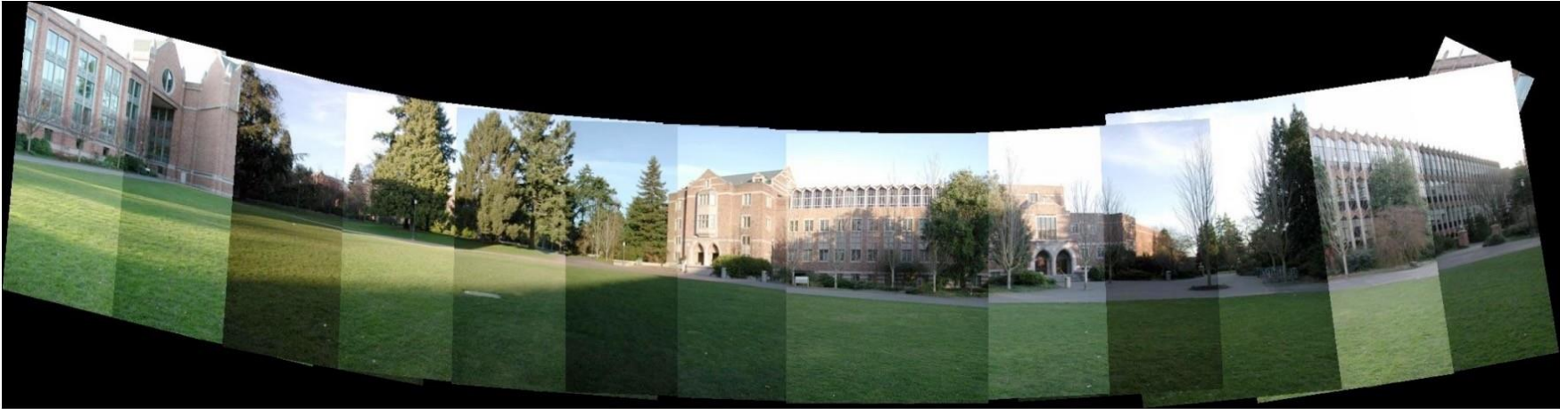
Application: Panorama

- Result



Application: Panorama

- Results





What we have learned today?

- Scale invariant region detection
 - Automatic scale selection
 - Difference-of-Gaussian (DoG) detector
- SIFT: an image region descriptor
- HoG: another image region descriptor
- Application: Panorama